Graph-Based Approaches and Functionalities in Retrieval-Augmented Generation: A Comprehensive Survey

ZULUN ZHU, College of Computing and Data Science, Nanyang Technological University, Singapore, Singapore
TIANCHENG HUANG, College of Computing and Data Science, Nanyang Technological University, Singapore, Singapore
KAI WANG, College of Computing and Data Science, Nanyang Technological University, Singapore, Singapore
JUNDA YE, Beijing University of Posts and Telecommunications, Beijing, China
XINGHE CHEN, College of Computing and Data Science, Nanyang Technological University, Singapore, Singapore
SIQIANG LUO, College of Computing and Data Science, Nanyang Technological University, Singapore, Singapore

Large language models (LLMs) struggle with the factual error during inference due to the lack of sufficient training data and the most updated knowledge, leading to the *hallucination* problem. Retrieval-Augmented Generation (RAG) has gained attention as a promising solution to address the limitation of LLMs, by retrieving relevant information from external source to generate more accurate answers to the questions. Given the pervasive presence of structured knowledge in the external source, considerable strides in RAG have been made to employ the techniques related to graphs and achieve more complex reasoning based on the topological information between knowledge entities. Although recent surveys summarize RAG pipelines, they largely restrict the role of graphs to knowledge-graph traversal, leaving the broader impacts of graph structure on retrieval, prompting, and pipeline control underexplored. This survey offers a novel perspective on the functionality of graphs within RAG and their impact on enhancing performance across a wide range of graph-structured data. It provides a detailed breakdown of the roles that graphs play in RAG, covering database construction, algorithms, pipelines, and tasks. Finally, it identifies current challenges and outline future research directions, aiming at inspiring further developments in this field. Our graph-centered analysis highlights the commonalities and differences in existing methods, setting the stage for future researchers in areas such as graph learning, database systems, and natural language processing.

This research is supported by the National Research Foundation, Singapore under its Frontier CRP Grant (NRF-F-CRP-2024-0005), and under its AI Singapore Programme (AISG Award No: AISG3-RP-2024-034).
This work was completed while Junda Ye was visiting Nanyang Technological University.
Authors' Contact Information: Zulun Zhu, College of Computing and Data Science, Nanyang Technological University, Singapore, Singapore; e-mail: ZULUN001@ntu.edu.sg; Tiancheng Huang, College of Computing and Data Science, Nanyang Technological University, Singapore, Singapore; e-mail: tiancheng.huang@ntu.edu.sg; Kai Wang, College of Computing and Data Science, Nanyang Technological University, Singapore, Singapore; e-mail: kai_wang@ntu.edu.sg; Junda Ye, Beijing University of Posts and Telecommunications, Beijing, Beijing, China; e-mail: jundaye@bupt.edu.cn; Xinghe Chen, College of Computing and Data Science, Nanyang Technological University, Singapore, Singapore; e-mail: xinghe001@e.ntu.edu.sg; Siqiang Luo (corresponding author), College of Computing and Data Science, Nanyang Technological University, Singapore, Singapore; e-mail: siqiang.luo@ntu.edu.sg.

ACM 0360-0300/2026/04-ART261
https://doi.org/10.1145/3795880

CCS Concepts: • **Information systems** → **Information retrieval**; • **Computing methodologies** → **Natural language processing;**

Additional Key Words and Phrases: Graph, large language model, retrieval-augmented generation.

ACM Reference Format:
Zulun Zhu, Tiancheng Huang, Kai Wang, Junda Ye, Xinghe Chen, and Siqiang Luo. 2026. Graph-Based Approaches and Functionalities in Retrieval-Augmented Generation: A Comprehensive Survey. *ACM Comput. Surv.* 58, 10, Article 261 (April 2026), 38 pages. https://doi.org/10.1145/3795880

1 Introduction

Large Language Models (**LLMs**) have demonstrated high effectiveness in a wide spectrum of human-centric tasks, including text summarization [45], question answering [39], machine translation [58], and code generation [118]. Early models like BERT [43] and RoBERTa [121] introduced advanced techniques for leveraging contextual embeddings, significantly enhancing performance of different natural language understanding tasks. Inspired by these fundamental works, models such as GPT-4 [142], LLaMA 2 [180], and more recently Deepseek-R1 [38], have set new benchmarks by utilizing transformer decoder-only architectures and scaling up model sizes. These LLMs excel at generating human-like responses, adapting to a wide range of domains, and supporting complex reasoning. All of these advancements have led to a growing consensus that LLMs represent a key pathway to **artificial general intelligence** (**AGI**) [24].

Despite these successes, LLMs face two major challenges: they are prone to factual errors and hallucinations, and they frequently struggle with real-world knowledge that inherently represents structured information [2, 105, 231]. The first challenge arises because the retraining of LLMs to incorporate new or domain-specific knowledge is computationally expensive [55], leaving models susceptible to producing logically coherent but factually incorrect inferences, which is also known as the hallucination problem [166]. For the second challenge, knowledge graphs like DBpedia [8], Wikidata [182], and Freebase [20] are primary examples of how entities and relationships are organized in a structured format, yet existing LLM-based approaches often fail to fully exploit these relational patterns, especially for complex or multi-hop reasoning tasks.

These two challenges call for retrieval mechanisms that enable LLMs to dynamically access **external** and **structured** knowledge during inference. **Retrieval-Augmented Generation** (**RAG**) helps solve the first challenges by retrieving relevant external information, thereby reducing hallucinations and enhancing factual accuracy [63, 82]. With respect to the second challenge, the integration of graph data management with RAG further enhances these capabilities by effectively organizing knowledge into organized forms such as knowledge graphs [66]. This integration captures complex relational patterns and supports multi-hop reasoning tasks such as node classification [32, 217], link prediction [42, 144], and subgraph extraction [68, 107, 123]. Consequently, graph-enhanced RAG provides a powerful approach to improving LLM's factual reliability and their ability to execute complex, structured reasoning tasks.

The application of graph techniques in RAG has rapidly expanded, leading several recent surveys on LLMs to discuss graph-based methods, such as Han's survey, Peng's survey in 2024 [147] and Zhang et al.'s more recent work in 2025 [218]. While prior surveys provide a useful synthesis of RAG architectures and applications with graphs, they are largely pipeline-centric and often conflate graph-based RAG with knowledge-graph traversal. Consequently, they devote limited attention to the underlying graph techniques, and provide mainly high-level rather than task-specific guidance for applying graph techniques in LLM pipelines. In contrast, this article fills this notable gap by providing a meticulous and detailed decomposition of the RAG framework,

broadening the definition of graph-based RAG and thoroughly examining the concrete graph techniques applied in representative works. We explicitly discuss the rationale behind integrating these graph methods into RAG systems and highlight their specific advantages in enhancing answer accuracy and efficiency. Furthermore, based on insights drawn from over 200 recent RAG studies, we systematically categorize the use of graph techniques across the core components of RAG—including database construction, retrieval or prompting algorithms, processing pipelines, and task scenarios. Unlike previous surveys [52, 63, 77, 147], this work offers an in-depth exploration from the viewpoint of graph data management with respect to database construction, query algorithms, prompt structure, and pipeline design, presenting novel insights into how detailed graph methods can significantly drive advancements in RAG research. The contributions of this work are summarized as follows:

- **Detailed and Case-by-Case Introduction of Graph Techniques in RAG:** We meticulously introduce concrete graph techniques employed in RAG, analyzing them case by case, unlike the brief and superficial descriptions provided in existing surveys. This systematic and detailed review clarifies both the rationale for selecting these methods and their practical benefits in enhancing accuracy and efficiency.
- **Novel Perspective and Taxonomy:** We introduce a novel perspective on the roles of graphs in RAG, encompassing database construction, algorithms, pipelines, and tasks. Additionally, we propose a taxonomy to categorize existing RAG methods and provide insightful discussion of their pros and cons.
- **Comprehensive Resource and Future Directions:** Based on an extensive review of over 200 studies, we provide a comprehensible resource to guide graph researchers in contributing to this evolving field. We introduce the representative works in detail to demonstrate the functionality of graphs to enhance the reasoning process of LLMs. This work lays the foundation for future advancements in areas such as graph learning, database systems, and natural language processing.

2 Comparison with Existing Surveys

Surveys of LLMs. A large number of surveys [17, 69, 79, 133, 137, 138, 201, 225] have explored LLMs from various perspectives over the years, reflecting their growing prominence in AI research. Early works [69, 201] provide comprehensive overviews of foundational models like BERT [43] and GPT-3 [23], detailing their architectures, design principles, and advancements compared to earlier models. These surveys emphasize the evolution of LLMs and their capacity to handle zero-shot and few-shot learning tasks, marking a shift from task-specific fine-tuning. Other studies [133, 164, 225] focus on specific technical aspects, such as pretraining methods, fine-tuning strategies, and alignment techniques for generating reliable outputs. They also address challenges like mitigating biases, ensuring factual accuracy, and aligning outputs with user intent.

Surveys of RAG. Building on the foundation of LLM surveys, an increasing number of works have concentrated on RAG, exploring its workflow and the integration of external knowledge to enhance generative accuracy. Some surveys [52, 63, 77, 82, 195, 213, 232] delve into the architecture of RAG, detailing how retrieval mechanisms interact with generative models to create a seamless pipeline for producing context-aware responses. These studies often highlight the modularity of RAG workflows, emphasizing the flexibility of incorporating different retrieval strategies, such as vector-based retrieval and hybrid methods, to optimize performance across diverse tasks. Other surveys [81, 155, 179, 221] prioritize addressing the hallucination problem, analyzing how external information retrieval mitigates factual inaccuracies, and ensures the generated responses align closely with real-world knowledge. They discuss the effectiveness of external retrieval in outputs of grounding models and reducing the risk of plausible but incorrect information generation. Additionally, some

works [223, 224, 228] explore the broader applications of RAG, demonstrating its utility in tasks like question answering, summarization, and domain-specific knowledge generation. However, given the importance of graphs as a fundamental structure in RAG, graph-related applications are not a central focus of these works.

Surveys of LLMs on graphs. Existing surveys of LLMs on graphs can broadly be categorized into two categories. The first category focuses on how LLMs can enhance prediction tasks on graphs by utilizing their ability to comprehend natural language [53, 93, 110, 119, 130]. These surveys examine techniques such as (i) feature augmentation, where LLMs enhance node or edge attributes; (ii) feature alignment, which bridges textual and graph representations; and (iii) structural enhancement, leveraging LLMs to model intricate graph relationships and topological patterns. These works emphasize the potential of LLMs in boosting the performance of tasks like node classification, link prediction, and graph-based recommendation systems. Surveys in another category [26, 145, 146] investigate how LLMs enhance traditional symbolic knowledge bases through three key approaches: (i) deploying LLMs as knowledge graph builders and controllers; (ii) leveraging structured knowledge to improve LLM pretraining; and (iii) enabling LLM-augmented symbolic reasoning to refine logical inference. With the approaches above, these methods showcase the potential of LLMs to enrich and expand the capabilities of knowledge graphs in both construction and reasoning tasks. Another important related work [147] examines the general workflow of how RAG leverages external knowledge graphs to enhance LLM predictions.

Comparison with existing surveys on graph-based RAG. Existing surveys on graph-based RAG [70, 147] provide a valuable synthesis of the RAG workflow and have helped stabilize terminology and component boundaries. At the same time, they are largely component-centric and tend to equate graph-based RAG with knowledge-graph traversal, which narrows the view of what graphs contribute to retrieval and reasoning. This perspective gives limited attention to analysis grounded in data-management practice and offers only modest, task-level guidance for graph researchers who wish to apply graph techniques in LLM-centric pipelines.

Our survey addresses these gaps along three axes while building on the strengths of prior work. First, it adopts a broader definition of graph-based RAG, encompassing graph-powered databases beyond curated knowledge graphs and clarifying how structural signals inform retrieval and prompting. Second, it provides a deeper analysis from the perspective of graph data management, treating retrieval, prompting, and pipeline topology as design problems with explicit tradeoffs in accuracy, latency, token budget, freshness, and provenance. Third, it serves as a comprehensible resource for graph researchers, offering task-wise guidance and decision criteria that indicate when graph structure materially improves outcomes and how to apply it in practice. Together, these contributions complement existing surveys by shifting the emphasis from pipeline components to the roles graphs play in RAG and by translating those roles into actionable methodology.

3 Foundations of Graph-Enhanced RAG

3.1 Overall Workflow of RAG

The standard RAG pipeline combines external knowledge retrieval with the generation abilities of LLMs, creating a robust pipeline for producing contextually accurate responses. We present an example for demonstration in Figure 1. In this scenario, a user query explores the development of the *general theory of relativity*, which involves background knowledge about scientists *Albert Einstein* and *Marcel Grossmann*. As this information may exceed the capacity of an LLM, RAG extends its scope by retrieving relevant knowledge from an external database. This process constructs a more informative prompt, thereby improving response accuracy. The complete workflow can be divided into the following four steps:

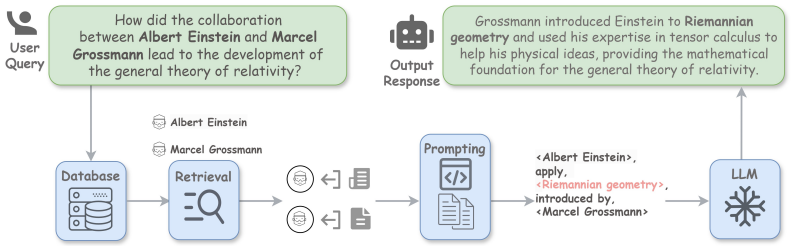

Fig. 1. Paradigm overview of RAG.

(a) User query. The workflow begins with a user-provided query, typically in natural language, such as a question or request for specific information. In our running example, the user initiates a dialogue to explore the development of the *general theory of relativity*. Additionally, the query includes a constraint specifying that the response should focus on the collaboration between *Albert Einstein* and *Marcel Grossmann*.

(b) Retrieval module. The query is passed to a retrieval system that interacts with an external knowledge database, such as a document corpus, knowledge graph, or vector store. Then, the retrieval system identifies and fetches the information most closely aligned with the query. The example query involves three key entities essential for answering the question: *general theory of relativity*, *Albert Einstein*, and *Marcel Grossmann*. The RAG system retrieves relevant information associated with these entities. In the context of a knowledge graph, the retrieved knowledge can be represented as triples, such as <*Marcel Grossmann*, introduced *Riemannian geometry* to, *Albert Einstein*>.

(c) Prompting module. The retrieved knowledge is transformed and integrated into the original query, ensuring that the model has access to up-to-date and domain-specific knowledge for generating accurate responses. Based on the key information retrieved to address the development of the *general theory of relativity*, the RAG system integrates external knowledge and reformats it to align with the LLM's input style. For example, it may generate a statement such as: "*Albert Einstein* applied *Riemannian geometry* in developing the *general theory of relativity*, a concept introduced to him by *Marcel Grossmann* during their collaboration." This prompting strategy helps the LLM more effectively process and incorporate new information naturally into its responses.

(d) Output response. Finally, the LLM leverages the enriched context to generate a response that is both accurate and highly relevant to the user's query. By incorporating external knowledge, the model not only captures the core rationale—such as the role of *Riemannian geometry* in the development of the *general theory of relativity*—but also structures the information in a more coherent and explanatory manner. This ensures that the response not only conveys factual accuracy but also presents the historical and conceptual development of the *general theory of relativity* in a clearer, more logically connected way.

3.2 RAG with Graph Data Management

Graphs, with their inherent ability to model complex relationships and dependencies, serve as powerful tools for structuring external knowledge and enabling sophisticated reasoning in RAG workflows. Existing literature [93, 147] primarily focuses on how LLMs enhance graph performance or provide a brief introduction of the RAG process involving graphs. However, several critical questions remain unanswered: *What benefits does the graph structure offer to RAG? Which graph patterns can enhance the effectiveness or efficiency of LLMs responses? What future directions should be pursued to develop graph algorithms that synergize with LLMs?* To address these compelling

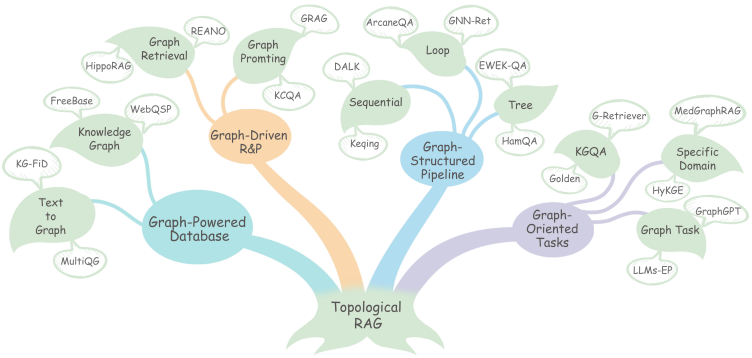

Fig. 2. RAG with graph data management.

questions, we approach this survey explicitly from the perspective of graph data management, systematically investigating how graph techniques enhance each core stage of the RAG process, as illustrated in Figure 2. Specifically, we highlight how graph data management methods significantly (i) improve knowledge base construction; (ii) optimize retrieval and prompting algorithms through graph-based indexing, querying, and reasoning; (iii) streamline data processing pipelines via graph-structured workflows; and (iv) effectively support graph-oriented tasks.

Graph-Powered Databases. In Section 4, we will explore the methods used to construct auxiliary databases for RAG. The discussion will be organized into two categories: existing knowledge graphs and the graph from the texts. For each category, we will analyze the representative graph techniques employed in existing literature and examine how they contribute to building effective auxiliary databases. This analysis will provide insights into how these databases are prepared to support subsequent retrieval tasks in RAG systems.

Graph-Driven R&P (retrieval and prompting). In Section 5, we will examine the graph algorithms used in RAG from two perspectives: graph **retrieval and prompting** (**R&P**). For graph retrieval, we will categorize existing graph techniques into non-parameterized methods and learning-based methods, analyzing their effectiveness and efficiency in retrieving relevant information. For Graph Prompting, we will explore two distinct classes: Graph-Structure Prompting and Text Prompting, focusing on how they transform graph information into formats that can be utilized by LLMs. For each category, we present concrete examples to illustrate the specific techniques and their practical applications.

Graph-Structured Pipelines. In Section 6, we categorize the pipelines in RAG into three distinct types: Sequential Pipeline, Loop Pipeline, and Tree Pipeline. This classification is based on the way where each method structures the overall pipeline, reflecting the underlying graph-like relationships between different module. By capturing the topological characteristics of these pipelines, we aim at shedding light on how their designs influence efficiency, scalability, and reasoning capabilities. Through case studies of representative projects, we will analyze how these different pipeline facilitate the integration of retrieval and generation, ultimately enhancing the performance of LLMs in diverse scenarios.

Graph-Oriented Tasks. Finally, in Section 7, we systematically categorize the applications of RAG into three key tasks: (i) **knowledge graph question answering** (**KGQA**) tasks, which involves responding to natural language queries using structured knowledge; (ii) graph-centric tasks (e.g., node classification, link prediction), leveraging reasoning property of LLMs to enhance graph reasoning; and (iii) domain-specific applications (e.g., biomedicine, finance), where graphs integrate domain knowledge to refine retrieval precision. For each category, we analyze how

Table 1. Summary of Commonly used KGs and the Methods Constructing KG

Categories	Pros	Cons	References
Existing KGs	*Broad coverage, reliable quality*	*General knowledge, less adaptable*	BioRED [125], QALD-9-plus [149], OpenbookQA [132], CREAK [141], TriviaQA [98], HotpotQA [205], Mintaka [162], MedQA [95], TUDataset [135], CWQ [172], Beyond I.I.D. [64], CommonsenseQA [173], SocialIQA [158], PIQA [19], RiddleSense [115], Freebase [88], ATOMIC [157], FactKG [101], MultiHop-RAG [177], T-REx [50], DBpedia [8], Yago [167]
Graphs generated from texts	*Domain-specific, easily updated*	*LM-dependent, computationally intensive*	GraphRAG [49], GRBK [41], ATLANTIC [136], GNN-Ret [113], HippoRAG [68], DALK [107], KGP [189], OpenSCR [72], MindMap [192], FABULA [154], GER [197], FoodGPT [152], ChatKBQA [124], MultiQG [104], HSGE [169], ReTraCk [29], RNG-KBQA [208], ArcaneQA [65], HybridRAG [159], EWEK-QA [39], KG-FiD [212], REANO [54], MedGraphRAG [193], MINERVA [37]

different RAG frameworks harness graph structures to optimize retrieval mechanisms, improve contextual relevance, and strengthen inferential accuracy.

4 Graph-Powered Databases

Database construction serves as a foundational step in the RAG paradigm, as it organizes and stores external knowledge to facilitate effective information retrieval. Specifically, the database in graph-based RAG captures entities and relationships from plain textual knowledge into a structured format, enabling efficient access to local (immediate neighbors or direct relationships) and global (overall connectivity or multi-hop paths) topological information. Formally, let $\mathcal{E}$ denote the set of entities and $\mathcal{R}$ denote the set of relationships; a particular fact is represented as a triplet: $\mathrm{t} = (e_h, r, e_t) \in \mathcal{E} \times \mathcal{R} \times \mathcal{E}$, where e_h and e_t denote the head and tail entities, respectively, and r denotes the relationship linking them. A graph database $\mathcal{G}$ thus consists of a collection of such triples: $\mathcal{G} = (e_h, r, e_t)$. In practice, various types of graph databases have been utilized in the RAG literature (summarized in Table 1), each offering distinct characteristics suited to different retrieval and reasoning tasks. In the following sections, we first introduce the existing graph databases and then discuss widely adopted techniques for generating such databases from textual data.

4.1 Existing Knowledge Graphs

4.1.1 A Unified View. In this section, we focus on existing knowledge graphs, which play a crucial role in many RAG systems by serving as structured repositories of factual knowledge. To enable efficient querying and traversal of entities and their relationships, many existing methods [10, 76, 87] directly retrieve knowledge relevant to the query from well-established knowledge graphs such as FreeBase [20], T-REx [50], and WebQSP [209]. These widely used databases offer comprehensive scopes for knowledge retrieval, encompassing a vast array of entities and relationships across diverse domains. For instance, T-REx [50] provides extensive alignments between natural language expressions and knowledge base triples, facilitating the integration of structured data with textual information. By organizing nodes and edges along with their textual attributes, these structured repositories allow RAG systems to efficiently access additional topological information, supporting advanced operations such as entity retrieval, relationship discovery, and complex multi-hop queries.

4.1.2 Existing Knowledge Graphs. **Freebase** [20] is a large-scale, structured knowledge database designed to organize and store general human knowledge. It combines the efficiency and scalability of structured databases with the rich semantic representation found in sources like Wikipedia [22], enabling flexible integration of diverse knowledge. By structuring information into tuples, Freebase facilitates efficient querying, entity linking, and knowledge retrieval across multiple domains. Freebase features an HTTP-based API powered by the **Metaweb Query Language** (**MQL**) [60], which facilitates intuitive object-oriented queries and schema evolution. Its complete normalization philosophy ensures unique global identifiers (GUIDs) for real-world entities, while its lightweight

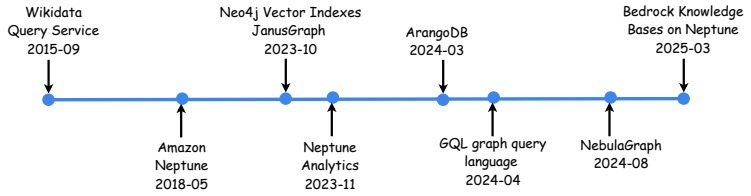

Fig. 3. Timeline of recent graph database.

typing system supports multiple data. In the context of RAG, Freebase serves as a powerful auxiliary database, enabling methods to retrieve structured knowledge for enhanced reasoning and answer generation. With its ability to handle vast datasets and diverse relationships, Freebase provides a robust foundation for integrating external knowledge into RAG systems, improving their capability to process complex queries and generate context-aware responses.

T-REx [50] is a large-scale dataset designed to align natural language from Wikipedia abstracts with **knowledge base** (**KB**) triples from Wikidata. Addressing the alignment problem, T-REx connects unstructured text with structured knowledge representations by providing 11 million triples aligned with over 3 million Wikipedia abstracts, covering more than 600 unique predicates. Its customizable alignment pipeline incorporates techniques like predicate linking, co-reference resolution, and distant supervision to create high-quality alignments. Compared to existing datasets, T-REx provides a larger scale, broader predicate coverage, and high accuracy. This dataset is instrumental in advancing tasks like relation extraction, KB population, and question answering, and serves as a foundational resource for RAG systems to retrieve structured knowledge and improve alignment for complex reasoning.

4.1.3 Industrial Graph Database for LLM. Recently, widely used graph databases have moved from early research backends to platforms that support LLM use cases with built-in embedding search, standard query languages, and managed GraphRAG pipelines. As shown in Figure 3, we list these important databases on a timeline: Wikidata's public query service as a common open graph backend;[1] Amazon Neptune as a managed graph service, later adding Neptune Analytics for large-scale graph and embedding analytics;[2] Neo4j with mature property-graph capabilities and native vector indexes for hybrid retrieval;[3] JanusGraph as common open-source choices for large or real-time workloads;[4] ArangoDB as a multi-model system that added vector indexes;[5] vendors beginning to productize GraphRAG (for example, NebulaGraph; Bedrock Knowledge Bases on Neptune);[6] and the GQL graph query language becoming an ISO standard that aligns vendor ecosystems.[7]

4.1.4 Trend of Graph Database Construction. In the LLM era, graph databases have evolved along four directions that directly address freshness, cost of construction, and LLM integration. First, *standardization* has begun to reduce vendor lock-in and smooth model portability. For example, the publication of graph query language (e.g., ISO/IEC 39075:2024,[8] which formalizes a property-graph

[1] https://query.wikidata.org/
[2] https://aws.amazon.com/cn/neptune/
[3] https://neo4j.com/release/neo4j-4-0/
[4] https://janusgraph.org/
[5] https://www.arangodb.com/docs/3.12/release-notes/
[6] https://www.nebula-graph.io/posts/announcing_nebulagraph_RAG
[7] https://www.iso.org/standard/76120.html
[8] https://opencypher.org/,https://www.iso.org/standard/76120.html

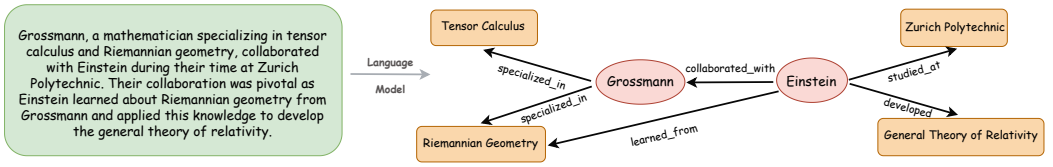

Fig. 4. From texts to knowledge graph.

data and query language and enables more consistent schemas, queries, and tooling across systems. Second, *hybrid graph–vector indexing* has become more popular so that semantic similarity can be combined with graph traversal. For instance, Neo4j made vector indexes generally available and documents RAG workflows on top of them,[9] while ArangoDB introduced native vector indexes and vector search functions of its declarative language.[10] Third, to lower *update latency and construction overhead*, **change-data-capture** (**CDC**) and streaming ingestion became first-class, such as Neo4j's CDC feature and Kafka connector strategies that propagate graph updates as events, reducing the gap between source changes and graph availability for RAG/agent pipelines.[11] Finally, the database construction increasingly blends *general encyclopedic KGs* with *domain KGs* for higher-quality grounding in graph-based RAG systems, a trend visible in community benchmarks such as GR-Bench [94] and GraphQA [74]. In summary, these updates reduce data-to-serving latency and integration cost, making graph databases easier to operate at scale while preserving the multi-hop structure and provenance required by LLM applications.

4.2 Graphs Generated from Texts

4.2.1 A Unified View. In addition to reusing existing knowledge graphs, we consider a broader class of graph-powered databases constructed from text. A common route is transforming plain text into a knowledge graph through artificial means, which is known as **open information extraction** (**OpenIE**) [6, 51, 226]. This process extracts key information—such as entities, relationships, and contextual meanings—from textual documents. An instruction-tuned language model then identifies these entities and their relationships to generate structured triples. As illustrated in Figure 4, given a paragraph describing the collaboration between *Albert Einstein* and *Marcel Grossmann*, the instruction-tuned LM captures the relationships among entities and converts them into structured triples. In longer contexts, multiple relationships emerge, linking entities to various neighbors and ultimately forming a complex graph.

Beyond vanilla knowledge graphs, we also include multiple forms graphs distilled from text: (1) document-level structures such as entity–sentence and sentence–sentence links (HippoRAG [68], ATLANTIC [136], GNN-Ret [113], OpenSCR [72], FiD [212], MedGraphRAG [193]); (2) hierarchical graphs with each node as a subgraph (DALK [107], GER [197]); (3) rule-based graph where each node aggregates multiple entities or facts consistent with a defined rule schema (FABULA [154], StarRAG [233], ReTraCk [29]). These constructions produce property graphs with additional information and traceable evidence, enabling unique feature extractions from the knowledge base.

4.2.2 Graph Generation Methods Based on Texts. **KG-FiD** [212] is a graph-based framework that constructs knowledge graphs directly from textual data to facilitate more effective information retrieval. Specifically, it begins by extracting text segments from the dataset and linking them to their associated entities. Using the BERT model for passage encoding, KG-FiD computes similarity scores

[9] https://neo4j.com/docs/cypher-manual/5/indexes/semantic-indexes/vector-indexes/
[10] https://docs.arangodb.com/stable/
[11] https://neo4j.com/blog/developer/change-data-capture-cdc-ga/

and reranks the passages within entities to enhance retrieval accuracy. Additionally, a **Graph Attention Network** (**GAT**) is utilized to propagate information across the graph, leveraging relationships between entities to retrieve relevant node information. This approach ensures that the graph representation is both contextually rich and informative, leading to improved retrieval outcomes.

MultiQG [104] proposes a method to answer multi-hop complex queries over knowledge bases by generating query graphs with constraints. The approach integrates constraints early in the process to guide the construction of query graphs, effectively reducing the search space. Candidate query graphs are ranked based on their embedding similarity with the question using a neural network, ensuring accurate alignment. By combining beam search and semantic matching, the method attains state-of-the-art performance on standard benchmark datasets like ComplexWebQuestions, demonstrating significant improvements in precision and F1 scores. This framework addresses the limitations of traditional RAG systems, which struggle to handle constraints effectively, improving accuracy in complex question.

4.3 Comparison between Existing and Generated Knowledge Graphs

Both existing knowledge graphs and text-derived graphs serve as essential components of graph-powered databases, each with distinct advantages and limitations. Existing knowledge graphs offer structured, high-precision information with efficient querying, making them well-suited for tasks requiring fast and reliable entity retrieval. However, they are inherently static and require manual updates to incorporate new knowledge. In contrast, graphs constructed from text provide greater adaptability by dynamically extracting relationships from unstructured data. While this approach enhances knowledge coverage and supports evolving information, it introduces potential workload due to reliance on language models and probabilistic extraction methods. Moving forward, hybrid approaches that integrate raw graphs with text-derived structures could offer a balanced solution, leveraging both efficiency and adaptability. Additionally, improving incremental learning for knowledge graph updates, enhancing entity-linking accuracy, and developing scalable retrieval mechanisms will be crucial for optimizing graph-powered databases.

5 Graph-Driven R&P

In the context of RAG, graphs play a crucial role in both R&P for LLMs– enabling the retrieval of structured knowledge and facilitating the prompting of LLMs. Specifically, *graph retrieval* refers to the algorithms designed to extract additional knowledge from the graph, which serves to enrich the reasoning capabilities of the LLMs. This process involves traversing the graph to find relevant nodes, edges, or subgraphs that provide valuable context or information necessary to enhance the accuracy and depth of the model's responses. On the other hand, *graph prompting* focuses on transforming the retrieved graph structure into a textual format that is easily understood by the LLMs. This transformation ensures that the complex relationships and connections within the graph are effectively communicated in natural language, enabling the LLMs to leverage the structured knowledge in its generative process. In the subsequent sections, we explore a range of graph-based retrieval and prompting techniques, highlighting representative approaches. The broader set of approaches, along with our classification, is presented in Tables 2 and 3.

5.1 Retrieval with Non-parameterized Algorithms

5.1.1 A Unified View. The non-parameterized methods in the retrieval process of RAG focus on extracting relevant knowledge from the graph using predefined rules, without involving any trainable parameters. Specifically, given the query q and the knowledge graph $\mathcal{G}$, the retrieval response R_q from the graph can be formulated as: $R_q = f(\mathcal{G}, e(q))$, where $e(\cdot)$ is the entity or relationship extraction function and $f(\cdot)$ is a deterministic function that follows predefined rules.

Table 2. Summary of Retrieval Algorithms

Retrieval Algorithms		Pros	Cons	References
Non-parameterized Algorithms	Deterministic Algorithms	*Precise, reliable*	*Intensive computation*	KG-GPT [100], GLBK [41], GRAG [76], HyKGE [90], Knowledge Solver [56], KnowledGPT [187], GGE [61], Engine [230], GraphBridge [190], MMGCN [191], NLGraph [184], GraphEval2000 [194]
	Probabilistic Algorithms	*Scalable, strong adaptability*	*Less precise, less interpretable*	HippoRAG [68], MindMap [192], OreoLM [78], MultiQG [104], MINERVA [37], MuseGraph [174], Walklm [175]
	Heuristic-Based Algorithms	*Efficient, scalable*	*Uncertain results depend on heuristics*	RA-SIM [148], KG-Rank [202], SKP [47], NuTrea [35], Zeshel [122], Conll [75], RNG-KBQA [208], ArcaneQA [65], HybridRAG [159],MedGraphRAG [193], TOG2 [128], DepsRAG [4], KELP [117], KGQA [163], Graph-LLM [32], GLBench [111]
Learning-based Algorithms	Convolutional-based Algorithms	*Efficient local structure modeling*	*Limited global information*	GNN-RAG [131], MHKG [57], RA-SIM [148], GenKGQA [62], GRAG [76], GNN-Ret [113], ETD [116], NuTrea [35], KAM-CoT [134], ConvE [42], REANO [54], SURGE [99], GraphGPT [176]
	Attention-Based Algorithms	*Global structure modeling*	*High computational cost*	ATLANTIC [136], OpenSCR [72], GGE [61], GER [197], HSGE [169], HamQA [48], KG-FiD [212], Fact [143]

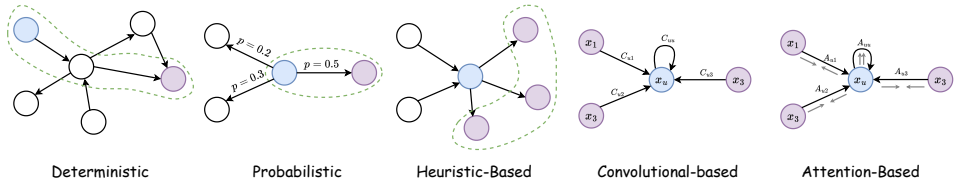

Fig. 5. Non-parameterized algorithms and learning-based for graph retrieval.

To minimize overlaps of these deterministic functions and ensure the uniqueness of each group, we categorize the methods into the following three classes: *Deterministic Graph Algorithms*, *Probabilistic Graph Algorithms*, and *Heuristic-based Graph Algorithms*. We outline the characteristics of these methods below and provide a visual representation in Figure 5.

— **Deterministic Graph Algorithms.** These algorithms focus on providing exact and reliable solutions to graph-related problems, ensuring correctness and precision. They prove especially effective for tasks requiring strict guarantees, such as subgraph isomorphism detection, shortest path computation, and graph traversals (e.g., Dijkstra's algorithm [44], Bellman-Ford algorithm [14]). Their main advantage lies in their accuracy, but they may incur high computational costs for large-scale graphs. Figure 5 illustrates an example of the exact path computed between two nodes, highlighting how deterministic algorithms ensure optimality in path finding.

— **Probabilistic Graph Algorithms.** These methods leverage probabilistic and statistical techniques to approximate graph properties or retrieve relevant information about entities and relationships. Instead of guaranteeing exact results, they offer efficient solutions for tasks where full precision is unnecessary or infeasible. Personalized PageRank, Markov Chains, and Monte Carlo methods are common examples, excelling in applications such as node ranking, influence estimation, and recommendation systems. These methods tradeoff determinism for scalability and adaptability in dynamic or uncertain environments. Figure 5 illustrates an example of a probabilistic random walk, where transitions between nodes occur with a

Table 3. Summary of Prompting Methods

Prompting Method	Pros	Cons	Reference
Topology-Aware Prompting	*Preserves structure, enables multi-hop reasoning*	*Complex formatting, hard to understand for LLMs*	LLaGA [28], GNN-RAG [131], G-Retriever [74], GRAPH-COT [94], ROG [126], RRA [198], MVP-Tuning [83], Keqing [183], GRAG [76], ETD [116], DALK [107], MindMap [192], Lark [36], REALM [229], HybridRAG [159], EWEK-QA [39], TOG2 [128], SURGE [99], GraphGPT [176], NLGraph [184], HyKGE [90], HamQA [48], GraphEval2000 [194], MuseGraph [174], GOT [16]
Text Prompting	*Simple implementation, compatible with LLM input format*	*Loses structured information, limited reasoning ability*	Talk [55], GPT4Graph [66], UniOQA [112], FABULA [154], ODA [171], KnowGPT [219], Kaping [10], MedGraphRAG [193], Golden-Retriever [5], KALMV [11], Fact [143], DepsRAG [4], KELP [117], KGQA [163], Graph-LLM [32], GLBench [111], Walklm [175]

probability p, demonstrating how these algorithms explore graph structures in a stochastic manner.

— **Heuristic-based Graph Algorithms.** This category mainly includes algorithms designed to provide efficient approximate solutions, especially in cases where exact methods are computationally prohibitive. By using heuristic-based strategies or domain-specific insights, they improve scalability while maintaining reasonable accuracy. Examples include greedy algorithms for approximate subgraph matching, K-hop sampling, and subgraph sampling techniques, which are widely applied in community detection, graph partitioning, and large-scale knowledge retrieval. These methods prioritize computational feasibility and speed over absolute correctness. Figure 5 presents a toy example of 1-hop sampling, demonstrating how nodes efficiently gather information from their immediate neighbors.

By removing the trainable parameters in the algorithms, non-parameterized methods are relatively efficient in identifying informative subgraphs that provide valuable context for reasoning. These methods are particularly effective in scenarios where the structure of the knowledge graph is well understood, and simple traversal or ranking techniques can yield meaningful insights. Their simplicity and lack of model parameters make them easy to implement and computationally lightweight, allowing for quick and direct retrieval of relevant graph-based information to support the reasoning of LLMs.

5.1.2 Methods Using Deterministic Graph Algorithms. **GLBK** [41] recognizes entities in the question, computes shortest paths that connect them in the knowledge graph, and retrieves text linked to the nodes and incident edges along these paths, which reveals indirect relations that support answer synthesis. **HyKGE** [90] aligns query and hypothesis entities to a medical graph and then collects three deterministic reasoning chains between anchor entities, namely simple paths, co-ancestor chains, and co-occurrence chains, followed by pruning that keeps the most relevant connections as retrieval evidence. **GNN-RAG** [131] first scores candidate answers with graph neural reasoning and then extracts the shortest paths from the question entities to these candidates so that the retrieved paths provide faithful and efficient multi hop evidence. **KG-GPT** [100] segments the question into triple level units, matches entities to the graph through semantic alignment, identifies the relations that connect them, and assembles a subgraph that serves as compact retrieval support for downstream reasoning.

5.1.3 Methods Using Probabilistic Graph Algorithms. Within the taxonomy in Table 2, probabilistic graph algorithms cast retrieval as stochastic evidence flow on the graph. **Walklm** [175] introduces targeted random walk sampling from query anchored nodes, which turns the visited trails into textual sequences and aggregates path evidence to rank candidate neighborhoods and passages. **HippoRAG** [68] illustrates diffusion-driven retrieval, where detected query entities are aligned to knowledge graph nodes and used as seeds for Personalized PageRank, which spreads

probability through nearby structure and then applies specificity-based reweighting to select high probability neighborhoods and surface the associated passages for multi-hop reasoning.

5.1.4 Methods Using Heuristic-based Graph Algorithms. **HybridRAG** [159] pairs vector search over the document store with a one-hop neighborhood walk from entities detected in the question, which enriches the retrieved context and improves answer quality while keeping the search path simple. **GRAG** [76] illustrates structured candidate selection, where K-hop ego graphs are ranked by embedding similarity to the query. Then, a learnable pruner suppresses low-value nodes and edges, and the top candidates are merged into a compact subgraph for generation, which preserves semantic focus and graph topology under tight runtime budgets.

5.2 Retrieval with Learning-based Algorithms

5.2.1 A Unified View. In contrast to non-parameterized approaches, the learning-based retrieval method can be formulated as a parameterized function $f_\theta(\cdot)$ that learns to retrieve relevant knowledge by optimizing an objective function: $R_q = f_\theta(\mathcal{G}, e(q))$, where θ represents the learnable parameters within $f_\theta(\cdot)$. These parameters are trained on a dataset using an optimization objective, enabling the retrieval mechanism to adapt dynamically to different data distributions and retrieval tasks. Unlike traditional rule-based retrieval, which relies on predefined heuristics, learning-based approaches can generalize across domains, capture complex semantic relationships, and improve retrieval relevance over time. Depending on how the system processes and extracts information from the knowledge graph, learning-based retrieval algorithms can be categorized into the following two major classes:

- **Convolutional-based Algorithms.** Convolutional-based algorithms are designed to exploit the structural properties of graphs by performing neighborhood aggregation through trainable network layers. These methods are inspired by convolutional operations in image processing but are adapted to graphs to capture local connectivity patterns and facilitate information propagation across nodes. A representative example is the **Graph Convolutional Network (GCN)** [102], which employs a layer-wise message-passing mechanism to iteratively aggregate features from neighboring nodes. As shown on the LHS of Figure 5, given the central node x_u and its neighbors $x_1 \sim x_3$, different weights C_{uu} and $C_{u1} \sim C_{u3}$ are applied to aggregate the features from each neighbor. This weighted aggregation allows the central node to incorporate diverse contextual information, leading to richer feature representations. By stacking multiple layers, these algorithms allow retrieval models to learn hierarchical representations, enhancing their ability to extract relevant knowledge from graph-structured data.
- **Attention-based Algorithms.** Attention-based algorithms in graph retrieval have gained significant interest due to their ability to dynamically prioritize the most relevant parts of the graph during learning. Unlike traditional approaches that treat all neighboring nodes equally, these methods employ attention mechanisms to assign varying importance weights (A_{uu} and $A_{u1} \sim A_{u3}$ in RHS of Figure 5) to nodes and edges, enabling the model to focus on key relationships that are most critical for a given task. A prominent example is the GAT [181], which introduces node-wise attention coefficients to aggregate information more selectively. This adaptive weighting mechanism allows attention-based retrieval models to capture long-range dependencies, refine contextual understanding, and improve retrieval precision.

5.2.2 Methods Using Convolutional-based Algorithms. **REANO** [54] is a framework designed to leverage structural information in knowledge graphs to enhance the retrieval of triple features by employing an L-hop graph convolution. Before the retrieval step, REANO uses the TAGME model [59] to extract entities and then builds relationships among these entities through both intra-context

and inter-context manners. Subsequently, an L-hop GNN is applied to update entity representations, allowing REANO to select the top-k triple candidates based on their similarity to the given question. By incorporating the topological structure over multiple hops within the knowledge graph, REANO gathers critical information needed to answer questions, effectively alleviating the reasoning burden of the answer predictor and improving the overall efficiency of the QA process.

SURGE [99] is a framework designed to extract context-relevant knowledge from knowledge graphs while enforcing consistency across facts to ensure that the generated responses faithfully reflect the retrieved knowledge corresponding to the question. To achieve this, SURGE leverages an existing edge message passing framework [97], which transforms the edges of the original graph into nodes within a dual hypergraph [161]. This transformation enables the use of existing node-level GNNs to represent the relationships within the original graph. Subsequently, SURGE samples negative subgraphs and employs contrastive learning, encouraging the model to generate responses that are consistent with the positive subgraph, thereby improving the fidelity of the generated content. By enforcing consistency through contrastive learning, SURGE ensures that its responses remain faithful to the retrieved knowledge, enhancing the reliability of the generated answers.

5.2.3 Methods Using Attention-based Algorithms. **HSGE** [169] studies the complex interactions between the entities of the knowledge base by reasoning in the history semantic graph, which is built by employing a pre-trained language model BERT [43] on conversation contexts. A fundamental method of HSGE to retrieve the structure information in the history semantic graph is to update the node embeddings with the attention-based module such as TransformerConv [165]. In order to further adapt to the change triggered by the new-coming conversations, HSGE encodes the position of each historical interaction and utilizes the attention mechanism again to aggregate the most relevant information for each mentioned entity in the query. Compared with concatenating all previous conversation turns, this paradigm is more computationally efficient, while still retaining key context information needed for effective question answering.

HamQA [48] intends to identify the importance of different relationships between the entities by emphasizing the information most relevant for addressing the question. Specifically, HamQA formulates the weights of each edge in the KG utilizing a learnable function combined with the entity representations. Moreover, considering the geometrically hierarchical features between different entities, HamQA further employs Hyperbolic distances [12] to measure the importance of different neighbors. These two strategies enable HamQA to effectively calculate the attention scores when incorporating the adjacent information and serve as the constraints to guide graph propagation toward more significant messages.

5.3 Comparison between Different Retrieval Methods

Non-parameterized and learning-based retrieval methods offer distinct advantages and tradeoffs in graph-driven retrieval. Non-parameterized approaches rely on explicit graph traversal, ranking, or sampling techniques, making them computationally efficient and interpretable. They work particularly well in structured knowledge graphs, where exact or approximate methods can effectively retrieve relevant subgraphs without requiring training data. However, their performance is often limited by predefined heuristics and an inability to adapt dynamically to different retrieval contexts. In contrast, learning-based retrieval methods leverage trainable models to capture richer graph representations through message passing or attention mechanisms, allowing for more adaptive and context-aware retrieval. These methods excel at modeling complex dependencies but require substantial computational resources and labeled data for training. Moving forward, future research may focus on hybrid retrieval strategies that integrate the efficiency of non-parameterized methods with the adaptability of learning-based approaches. Additionally, improving self-supervised

learning for graph retrieval can reduce dependence on labeled data, enhancing the availability in real-world applications.

5.4 Topology-Aware Prompting

5.4.1 A Unified View. Topology-aware prompting captures the topological essence of a knowledge graph by explicitly encoding nodes, edges, and their relationships into structured prompts that models capable of directly processing structured inputs can readily interpret. These featured prompting methods can be divided into following categories: (1) path prompting that encodes relation chains or shortest paths to expose composition (GRAPH-COT [94], ROG [126], TOG2 [128], MindMap [192]); (2) subgraph prompting that linearizes compact subgraphs for information summarization (G-Retriever [74], DALK [107], GRAG [76], LLaGA [28]); (3) rule-aware prompting that introduces relation types and declarative constraints to guide reasoning (HyKGE [90], HamQA [48]).

Instead of providing simple textual descriptions, these prompts explicitly represent relationships using structured formats such as triple statements (e.g., "*(Albert Einstein, born in, Ulm)*") or relational paths (e.g., "*Paris → capital of → France → located in → Europe*"). A key motivation behind graph-structure prompting is to allow the model to reason more effectively about multi-hop and complex relational patterns [128]. For instance, rather than treating the graph as a mere collection of facts, the model can examine path-based relationships to draw inferences. This is especially valuable in scenarios where the desired answer depends on understanding connections among multiple entities or interpreting intricate graph substructures. Moreover, this approach can boost explainability by making the model's reasoning process more transparent. When each relationship or path is clearly defined, users can trace how the system derived its conclusions, thereby increasing trust and clarity in the model's output. In summary, graph-structure prompting offers a powerful mechanism for harnessing a graph's rich relational data, allowing RAG-based systems to deliver more reliable answers and deeper insights.

5.4.2 Methods Applying the Topology-Aware Prompting. **Keqing** [183] utilizes graph-structured prompting to enhance KGQA tasks by guiding LLMs through complex multi-hop questions. It achieves this by employing two distinct prompting strategies to help LLMs understand and reason about the logical relationships in retrieved triplets from a KG. The first strategy explicitly explains the triplet structure in the prompt, detailing the format as *(subject, relation, object)*, and ensuring that responses align strictly with the entities within the triplets. The second strategy converts triplets into plain text by serializing their components, making them more digestible for fine-tuned models like LLaMA 2 [180]. These graph-structured prompts act as a **chain-of-thought** (**COT**) mentor for LLMs, enabling them to decompose complicated queries into simpler sub-questions, retrieve relevant entities through logical chains, and generate accurate and interpretable responses. By leveraging the structured nature of KGs and tailored prompting techniques, Keqing demonstrates improved reliability and interpretability in KGQA tasks, highlighting its potential as a scalable solution for knowledge-intensive reasoning.

Both **RoG** [126] and **ToG** [170] generate reasoning paths from KGs to enhance RAG prompting by structuring retrieval and reasoning. RoG focuses on generating relation-grounded paths, ensuring faithful and structured reasoning by retrieving valid reasoning paths from KGs. This approach enhances accuracy and consistency while providing interpretable results that improve trust and understanding. Similarly, ToG identifies relevant entities from a query and explores the KG by searching for meaningful triples. During reasoning, the LLM evaluates the retrieved triples and selects the most valuable ones to construct a reasoning path. This method offers explicit and editable reasoning paths, improving explainability and allowing users to trace and correct model

outputs when necessary. Both approaches strengthen RAG by integrating structured retrieval with reasoning, ensuring more coherent, interpretable, and contextually relevant responses.

5.5 Text Prompting

5.5.1 A Unified View. In the cases where the machine struggles to understand the graph-structure prompt, text prompting involves transforming the structured graph knowledge with language models into a linear, textual representation that the LLMs can easily understand. After graph retrieval, the nodes, relationships, and properties are converted into descriptive sentences or paragraphs that capture the essence of the original graph while making it compatible with natural language input formats.

The focus of text prompting is on providing a human-like narrative that conveys the important details of the graph. Text prompting ensures that even complex graph structures are expressed in a format that a typical LLMs can understand, thus allowing the LLMs to utilize the graph-based information without requiring specialized graph-processing capabilities.

5.5.2 Methods Applying the Text Prompting. **MedGraphRAG** [193] follows a general, reusable pattern of graph-based RAG: graph building, graph retrieval, prompting, and refinement. In the build step, an LLM reads medical documents, extracts entities, links relations, and keeps short evidence or definition snippets so that each node and edge is grounded in source text. At query time, the system embeds the user question and graph entities, selects the top-k most similar entities, and expands a small working subgraph around them for focused context. The model then generates a draft answer from this subgraph and iteratively improves it by feeding the draft back into the LLM for a few rounds, stopping at a preset threshold and summarizing low-value regions to remain efficient. This simple loop—evidence-linked graph construction, embedding-based top-k retrieval with local expansion, and short refinement—captures a general template for reliable, source-aware graph-grounded QA in high-stakes domains.

KCQA [163] builds a framework that efficiently retrieves and prompts knowledge from a KG to answer questions. Initially, KCQA estimates the distribution of relations in the KG using the Rigel model [140] and subsequently predicts relevant triples within two hops. To focus on the most relevant information, it retains only the top-K triples of the estimated relations, achieved through a Hadamard (element-wise) product to extract a weighted vector representation of the triples. These selected triples are then converted into natural language for prompting. In the prompting process, KCQA also considers inverse triples and utilizes a unified template to compose the prompt, which is then fed as the input into a language model generating an answer.

5.6 Comparison between Different Prompting Methods

The strengths and limitations of topology-Aware prompting and text prompting map naturally to task demands. For multi-hop question answering [177], personalized recommendation [40], temporal reasoning [233], and program synthesis over graphs [65], topology-aware prompting is preferable because it preserves edge types and path structure, enforces constraints, and yields auditable chains. However, as shown in Table 3, it brings heavy formatting overhead, larger token budgets, schema canonicalization burdens, and can be harder for general LLMs to parse, making it brittle under noisy graphs [69]. By contrast, for abstractive summarization over documents attached to the graph [49], for broad retrieval where lexical recall dominates [35], and for user-facing explanations that prioritize fluency [187], text prompting is the better choice. It is simple to implement, aligns with standard LLM inputs, remains robust under schema drift, and achieves lower latency, but it sacrifices explicit path identity, weakens constraint satisfaction, and limits compositional reasoning depth [130].

Table 4. Summary of Graph-Structured Pipelines

Pipelines	Pros	Cons	Reference
Sequential Pipeline	*Simple and efficient execution*	*Fixed order, no refinement or error correction*	GNN-RAG [131], GrapeQA [178], SR [216], RRA [198], KG-GPT [100], QA-GNN [207], GLBK [41], ATLANTIC [136], UniOQA [112], GRAG [76], HyKGE, [90], KG-Rank [202], KnowledGPT [187], ETD [116], HippoRAG [68], DALK [107], KnowGPT [219], Engine [230], GraphBridge [190], ChatKBQA [124], NuTrea [35], and so on
Loop Pipeline	*Iterative refinement, error correction*	*Higher computational cost, low efficiency*	GraphRAG [49], GRAPH-COT [94], KnowledgeNavigator [67], KD-COT [185], KG-Agent [86], GNN-Ret [113], Knowledge Solver [56], KGP [189], FABULA [154], ODA [171], QR-LLM [129], MufassirQAS [3], ArcaneQA [65], MedGraphRAG [193], KALMV [11]
Tree Pipeline	*Parallel exploration, multi-path reasoning*	*Higher computational cost*	EWEK-QA [39], HamQA [48], SURGE [99], TOG2 [128], GOT [16], Beyond-CoT [206]

6 Graph-Structured Pipelines

6.1 A Unified View

Based on the pipeline of RAG, we can visualize the different steps in the pipeline as nodes, and the processes or transitions between them as edges. Each component in the RAG system can be represented in this graph-based structure, and the relationships between components can be defined based on their interactions. Particularly, we formulate each specific step as a node in the path of the directed graph as: (i) e_1: user query; (ii) e_2: graph retrieval; (iii) e_3: graph prompting; (iv) e_4: large language model generation; (v) e_5: output response. Moreover, we define the edge $r_{i,j}$ (from node e_i to node e_j) as: (i) $r_{1,2}$: passing the user query to the graph retrieval step; (ii) $r_{2,3}$: processing the graph retrieval output for prompting; (iii) $r_{3,4}$: passing the graph-structured or textual prompt to the LLMs; (iv) $r_{4,5}$: generating the final output response from LLMs.

In this configuration, the various steps and their transitional passages form a natural topological structure. As illustrated in Figure 6, we categorize the existing pipelines into three main types. (i) **Sequential Pipeline**. In reference to the figure, the sequential pipeline follows the path $e_1 \xrightarrow{r_{1,2}} e_2 \xrightarrow{r_{2,3}} e_3 \xrightarrow{r_{3,4}} e_4 \xrightarrow{r_{4,5}} e_5$, mirroring the standard RAG paradigm shown in Figure 1. This approach represents a linear flow, wherein each step immediately succeeds the previous one, and no node is revisited. (ii) **Loop Pipeline**. In this variant, certain stages incorporate feedback mechanisms or iterative processes. For instance, the graph retrieval step may be repeated if the initial retrieval fails to supply adequate context, or the LLMs might require additional prompts to refine their responses. We capture this behavior with a cyclic graph. In Figure 6, the final output can also be routed back to earlier steps, either merging with the user query ($e_5 \xrightarrow{r_{5,1}} e_1$) or informing another retrieval phase ($e_5 \xrightarrow{r_{5,2}} e_2$). (iii) **Tree Pipeline**. This paradigm is designed for scenarios in which multiple components of the pipeline can execute in parallel. The graph splits into multiple branches, allowing different retrieval or prompting strategies to occur simultaneously. In the following sections, we introduce representative methods employing these different pipeline designs and summarize a widely investigated classification in Table 4.

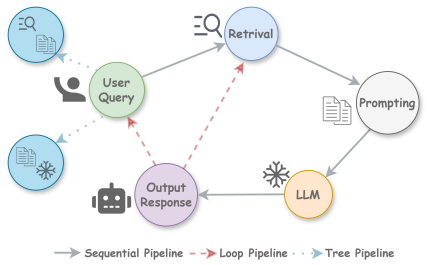

Fig. 6. Pipeline formulation with graph structure.

6.2 Controllers of Pipeline Implementation

The *controller* is the decision-making mechanism that formalizes our pipeline topologies at inference time, which chooses when to retrieve, whether to iterate or branch, when to verify and revise, and when to stop. Different controller families emphasize sequential, iterative loop, and branching tree pipelines, each with distinct tradeoffs among accuracy, latency, and computational cost. In what follows, we present four widely adopted controller families in RAG systems and show how representative methods instantiate each family within our pipeline design.

Reinforcement-Learning Controllers. A **reinforcement-learning** (**RL**) controller is a learned policy that, given the current reasoning state, selects the next pipeline action (retrieve, expand, verify, revise, or stop) to maximize a reward that tradesoff answer quality and computational cost. For example, RAG-RL [80] post-trains the generator with RL so it learns to select and cite useful contexts from larger retrieved sets, which tightens the coupling between retrieval and generation and effectively learns the loop and tree branching rules for exploration and pruning. Multi-module RL has also been used to coordinate several RAG components jointly (query rewriter, retriever, selector, generator), as in MMOA-RAG [30], which frames the pipeline as cooperative agents optimized toward a unified reward and thus implements a learned loop controller and branch selection over candidate contexts.

System-2 Slow-Thinking Controllers. A system-2 controller is an inference-time search policy that allocates more computation than a single forward pass, which naturally realizes loop and tree pipelines without updating model weights. For instance, Self-Ask [151] decomposes a question into sub-questions, interleaves retrieval during the reasoning process, and thus implements a loop that alternates "ask–retrieve–answer" over a constrained subgraph. FLARE [92] generalizes this idea to long-form generation by forecasting upcoming content, triggering retrieval only when predicted segments are low confidence, and regenerating with fresh evidence.

Verifier-Guided Controllers. A verifier-guided controller attaches an external evaluator that scores partial solutions or retrieved sets and feeds decisions back to the pipeline, thereby governing revision and selection. Self-RAG [7] trains a model to retrieve, generate, and *critique* using reflection tokens. During inference, it adaptively decides whether to retrieve again, accept or revise a segment, and produce citations, which instantiates a loop controller with verify–revise cycles over the current subgraph. CRAG [200] adds a lightweight retrieval evaluator that estimates the quality of retrieved documents and triggers corrective actions, such as re-retrieval or web fallback, effectively steering the loop and pruning poor branches in a tree of candidate evidence.

Budget-/Cost-Aware Controllers. A budget-aware controller explicitly optimizes an accuracy–latency objective by selecting per-query computation and retrieval budgets, thereby choosing whether a sequence suffices or whether a loop or tree is warranted.Adaptive-RAG [84] predicts query complexity and selects among no-retrieval, single-step retrieval, or iterative retrieval strategies, implementing a controller that upgrades a sequence pipeline into a loop only when complexity demands. Conversational gating methods such as RAGate [186] estimate, at each turn, whether external augmentation is needed and hence decide to skip or invoke retrieval, providing a cost-aware switch between sequence and loop behavior under compute or latency constraints.

6.3 Methods Applying the Sequential Pipeline

Keqing [183] adopts a sequential pipeline to address complex multi-hop question answering. It decomposes complex queries into simpler sub-questions using predefined templates. These sub-questions are then aligned with logical chains on a **knowledge graph** (**KG**), guiding the retrieval of candidate entities through multi-hop reasoning. A candidate reasoning module evaluates and selects the correct entities from the KG. Finally, the system generates responses by aggregating reasoning paths and answers to provide an interpretable and precise answer to multi-hop questions.

QA-GNN [207] integrates LMs and KGs for question answering with a sequential pipeline. The process begins by encoding the QA context including the question and answer options with an LM and retrieving a KG subgraph relevant to the question. A joint "working graph" is constructed by connecting the QA context node to entities within the KG subgraph. To enhance reasoning, the KG nodes are scored for relevance based on their alignment with the QA context using LM-based scoring. An attention-based GNN performs iterative message passing on this working graph, updating both KG entities and QA context representations. Finally, predictions are made by aggregating the LM output, updated node features, and the working graph representation, achieving accurate and explainable results.

DALK [107] makes LLMs and KGs benefit from each other through an elaborate sequential pipeline design. At the beginning of the pipeline, LLMs process the scientific corpus and construct two domain-specific KGs from the related literature via pair-wised relation extraction and generative relation extraction, respectively. This part of the pipeline can be seen as LLMs benefit KGs. Then, DALK utilizes a coarse-to-fine sampling method and a retrieval approach to select knowledge from KGs. The selected knowledge as well as the domain questions are sent to LLMs to generate accurate answers, therefore realizing that KGs benefit LLMs.

6.4 Methods Applying the Loop Pipeline

RGNN-Ret [113] tackles multi-hop reasoning questions by iteratively decomposing the reasoning process into manageable steps. It incorporates a self-critique mechanism that prompts LLMs to generate sub-questions for each reasoning step. After answering a sub-question with retrieved passages, the LLM evaluates whether the accumulated evidence is sufficient to generate a final answer or if further reasoning steps are required. This iterative process enables the system to dynamically adapt to the complexity of the query. To enhance the retrieval quality across reasoning steps, a **Recurrent Graph Neural Network** (**RGNN**) is used. The RGNN integrates the graphs of passages from previous reasoning steps, establishing connections between retrieved passages to ensure that relationships between sub-questions and evidence are maintained. By combining semantic distances and contextual information across steps, the RGNN reduces the impact of incorrect sub-questions and improves retrieval accuracy for supporting passages. The iterative loop continues until the self-critique determines that enough evidence has been collected to answer the question.

GRAPH-COT [94] provides a benchmark to enhance LLM reasoning on graphs through iterative steps of reasoning, interaction, and execution. At each iteration, the LLM identifies the required information and formulates graph interactions to retrieve relevant data, such as nodes or relationships. These interactions are executed on the graph, and the results are fed back into the reasoning process. This cycle repeats until the LLM concludes the reasoning task and produces the final answer. By iteratively refining its understanding of graph structures, GRAPH-COT effectively handles multi-hop reasoning tasks, ensuring accurate and explainable results in graph-based question answering.

6.5 Methods Applying the Tree Pipeline

SURGE [99] effectively leverages a tree pipeline by structuring its retrieval and generation processes into multiple parallel branches, allowing different retrieval and prompting strategies to execute simultaneously. Instead of sequentially retrieving a single set of facts, SURGE retrieves multiple context-relevant subgraphs in parallel, each representing different aspects of the dialogue history. These subgraphs are processed simultaneously through distinct embedding pathways, ensuring diverse knowledge representations. The model then applies its invariant graph encoding technique to each retrieved subgraph independently, maintaining permutation and relation-inversion invariance across multiple branches. Additionally, during response generation, SURGE integrates these parallel knowledge streams using graph-text contrastive learning, ensuring that the generated

response remains consistent across all retrieved knowledge sources. This parallel execution not only enhances efficiency but also enables the model to synthesize information from multiple perspectives, improving response accuracy and informativeness in knowledge-grounded dialogue systems.

GoT [16] effectively leverages a tree pipeline by expanding an initial prompt into multiple parallel branches of "thoughts," each branch pursuing a distinct partial solution under a predefined Graph of Operations. Rather than committing to a single chain, GoT repeatedly applies a generate–score–select routine: candidate branches are spawned with a controllable branching factor, locally validated and scored, and then pruned via a keep-best policy to preserve only the most promising subtrees. This staged branching decomposes the task into tractable subproblems that can be solved concurrently within separate branches, while refinement loops allow individual branches to self-correct without restarting the whole pipeline. Crucially, GoT then fuses the surviving branches using explicit aggregation operators, turning the pure tree into a directed acyclic workflow that consolidates partial results into a coherent final output. By combining parallel expansion, principled pruning, iterative refinement, and late-stage aggregation, GoT maintains the efficiency benefits of a tree pipeline across the entire process while achieving higher solution quality and better cost–latency tradeoffs than sequential methods.

Beyond-CoT [206] leverages a tree-style pipeline by expanding a single query into multiple concurrent branches that persist across two stages—rationale generation and answer inference. First, an **Extract–Cluster–Coreference** (**ECC**) routine builds a branched thought graph from the input and optional image caption, effectively fanning out the problem into many node-level sub-traces. Each branch is then encoded in parallel: text via a Transformer encoder, the thought graph via a graph attention network, and vision via a dedicated encoder. Cross-attention aligns tokens with graph nodes, and a gated-fusion layer aggregates these parallel branches into a unified representation that the decoder uses to produce rationales. In stage two, the pipeline re-branches by concatenating the predicted rationales back to the input, reconstructing the thought graph, and repeating the same parallel encoders, alignment, and gated fusion before decoding the final answer. This end-to-end design preserves the efficiency and diversity benefits of a tree pipeline—branching for exploration, parallel encoding for coverage, and late fusion for consolidation—while generalizing the tree into a graph to capture non-linear dependencies.

6.6 Comparison between Different Pipelines

Different pipelines offer varying degrees of flexibility and efficiency, influencing how information flows through RAG systems. Sequential pipelines provide a straightforward, stepwise execution, ensuring clarity and simplicity but limiting adaptability when intermediate refinements are needed. In contrast, loop pipelines introduce feedback mechanisms, allowing iterative refinement of retrieved knowledge or prompt modifications, making them well-suited for tasks requiring multiple rounds of reasoning. However, they can introduce computational overhead due to repeated processing. Tree pipelines, on the other hand, enable parallel execution of different retrieval or prompting strategies, improving efficiency in handling diverse queries but potentially increasing system complexity. A future trend of pipelines is moving toward adaptive pipeline selection, where the system dynamically determines the optimal pipeline structure based on the query's complexity and reasoning needs [91]. This strategy can enable more efficient, context-aware retrieval and generation while minimizing unnecessary computational overhead.

7 Graph-Oriented Tasks

7.1 KGQA Tasks

7.1.1 A Unified View. KGQA targets answering natural language queries by leveraging the structured information encoded in KGs. Unlike traditional question-answering methods that rely

Table 5. Summary of Different Tasks

Tasks	Reference
KGQA Tasks	HSGE [169], ReTraCk [29], RNG-KBQA [208], ArcaneQA [65], EWEK-QA [39], HamQA [48], KG-FiD [212], Golden-Retriever [5], SURGE [99],KELP [117], Fact [143], TOG2 [128], MINERVA [37], KALMV [11],REANO [54],LLaGA [28], GNN-RAG [131], G-Retriever [74], GraphRAG [49], ROG [126], SR [216], RRA [198], KnowledgeNavigator [67], KG-GPT [100], KD-COT [185], QA-GNN [207], MVP-Tuning [83], StructGPT [85], MHKG [57], MHGRN [25], Temple-MQA [33], KagNet [114], KG-Agent [86], GenKGQA [62], GLBK [41], Keqing [183], UniKGQA [87], GRAG [76], Knowledge Solver [56], WebQSQ [209], MetaQA [220], PullNet [168], KnowledGPT [187], ETD [116], DecAF [211],HippoRAG [68], GGE [61], NSM [73], SKP [47], KnowGPT [219], Difar [9], Kaping [10], NuTrea [35], OreoLM [78]
Graph Tasks	GPT4Graph [66], GraphText [222], LLaGA [28], GRAPH-COT [94], Engine [230], GraphBridge [190], MMGCN [191],ConvE [42], Graph-LLM [32], GraphGPT [176], NLGraph [184], GLBench [111], GraphEval2000 [194], MuseGraph [174], Walklm [175]
Domain-specific Tasks	GLBK [41], ATLANTIC [136], HyKGE [90], KG-Rank [202], DALK [107], KGP [189], MindMap [192], FABULA [154], FoodGPT [152], KAM-CoT [134], MufassirQAS [3], REALM [229], MEDQA [95], HybridRAG [159], MedGraphRAG [193], DepsRAG [4]

The brief introduction of the listed methods can be found in the online appendix [1].

solely on text-based retrieval, KGQA focuses on grounding its reasoning in the relationships and entities represented within a knowledge graph. By utilizing the rich semantic structure of KGs, which capture entities, their attributes, and the interrelationships between them, KGQA systems aim at providing precise and contextually accurate answers. This task not only requires understanding the natural language query but also necessitates navigating the graph's structure to extract relevant facts, perform logical inferences, and deliver a well-supported response. In the ensuing section, we offer in-depth discussions of select noteworthy works, and a more extensive overview of additional works can be found in Table 5. To better summarize these methods, we leave the brief introduction of them in the Appendix [1].

7.1.2 Methods Targeting the KGQA Tasks. **G-Retriever** [74] is designed to improve KGQA by addressing challenges in reasoning, scalability, and accuracy for real-world textual graphs. It introduces a new benchmark of graph-based question answering, enabling models to handle complex, multi-domain questions beyond basic graph reasoning tasks. A key innovation of this benchmark is formulating the retrieval of subgraph as a Prize-Collecting Steiner Tree optimization [18] problem, ensuring relevant information is extracted while maintaining explainability. G-Retriever integrates GNNs and LLMs for fine-tuned reasoning and achieves superior performance across domains such as knowledge graphs, scene graphs, and common-sense reasoning. By offering a conversational interface for graph queries, it advances the explainability, efficiency, and usability of KGQA tasks. This makes it a benchmark in advancing graph-based question-answering systems.

GraphRAG [49] enhances KGQA by integrating traditional RAG with graph-based summarization, addressing global queries that traditional RAG struggles to resolve. It builds an LLM-derived knowledge graph that uses community detection algorithms, such as Leiden, to group related entities and relationships into modular communities. These communities are pre-summarized, allowing for efficient multi-hop reasoning and improved retrieval accuracy. For answering queries, partial responses from community summaries are combined into a global answer through a map-reduce approach. This method improves comprehensiveness, diversity, and scalability, enabling precise, contextually rich responses for large-scale datasets with significantly reduced computational overhead.

7.2 Graph Tasks

7.2.1 A Unified View. By incorporating LLMs into graph task execution, the retrieval of relevant subgraphs or neighboring nodes can improve the accuracy of tasks like node classification, link

prediction, and graph classification. These tasks benefit from the reasoning capabilities of LLMs, which can understand complex relationships within the graph. Specifically, by utilizing the LLM's ability to process and reason over graph structures, predictions become more accurate, allowing for better insights and performance across a variety of graph-based applications.

7.2.2 Methods Targeting the Graph-centric Tasks. **RAGRAPH** [89] performs node classification, link prediction, and graph classification tasks. By retrieving and leveraging the most relevant subgraphs, RAGRAPH passes information within subgraphs to enhance the representation of the center node, facilitating various graph learning tasks. **LitFM** [215] considers link prediction (citation link prediction, article recommendation) and text generation (title generation, abstract completion, citation sentence generation) tasks.

GraphGPT [176] aims at aligning LLMs with graph learning tasks through a graph instruction tuning paradigm. The framework focuses on integrating graph structural knowledge with LLMs to enhance generalization in both zero-shot and supervised settings. GraphGPT includes a text-graph grounding component that aligns graph structures with natural language, allowing LLMs to comprehend complex graph structures. This alignment is achieved through a dual-stage instruction tuning process: first, self-supervised instruction tuning is employed to provide structural knowledge, and second, task-specific instruction tuning is used to enhance adaptability across various graph tasks, including node classification and link prediction. GraphGPT also leverages COT distillation to improve step-by-step reasoning abilities, making it effective at addressing the challenges of zero-shot learning in graph-based tasks.

7.3 Domain-specific Tasks

GraphRAG can be effectively employed in diverse domains such as academia, e-commerce, scientific literature, healthcare, and legislation due to its ability to integrate powerful language generation models with retrieval-based knowledge.

Healthcare. Existing literature leverages RAG to efficiently summarize long-form medical documents, enhancing LLMs' ability to generate accurate and evidence-based responses. For instance, MedGraphRAG [193] serves as a dedicated framework for utilizing graph-based RAG in healthcare applications. Its retrieval process involves generating a tag-summary on the user query, identifying the most relevant graph through similarity-based selection, and refining responses. HyKGE [90] is a framework leveraging RAG based on knowledge graphs in LLM-empowered medical applications. To improve medical consultation quality, HyKGE integrates a granularity-aware reranking module to eliminate noise while preserving diversity-relevance balance in retrieved knowledge. REALM [229] enhances the clinical predictive capabilities by integrating RAG with multimodal **Electronic Health Records** (**EHR**). It enhances the utilization of EHR data in healthcare and reconciles it with the intricate medical context necessary for educated clinical forecasts.

Scientific Literature. By harnessing the capabilities of GraphRAG, it becomes possible to navigate the vast expanse of scientific literature, extracting key insights and condensing complex ideas and high-quality knowledge. For example, DALK [107] constructs a specific knowledge graph sourced from Alzheimer's Disease scientific literature. The essential knowledge is filtered through a coarse-to-fine sampling algorithm so that it can offer valuable insights. ATLANTIC [136] introduces a retrieval-augmented language model that incorporates structural awareness for interdisciplinary scientific tasks, integrating heterogeneous document graphs to capture structural relationships among scientific documents. By fusing textual and structural embeddings, ATLANTIC enhances the retrieval of coherent and faithful passages. KGP [189] creates a KG over multiple kinds of literature and employs an LLM-guided traversal agent to retrieve and synthesize relevant information for answering complex queries.

Table 6. Evaluation of Different Tasks

Tasks	Metrics	Benchmark datasets
KGQA Tasks	Exact Match, F1 Score, SCR, Accuracy, ROUGE, BLEU, METEOR, TTFT	SimpleQuestion [21], WebQ [15], CWQ [172], MetaQA [220], CronQuestion [160], Forecast [46], MultiTQ [31], GraphQA [74], GRBENCH [94], CRAG [204], NQ [103], KILT [150], MultiHop [177], CRUD-RAG [127], CREAK [141], HotpotQA [205], TriviaQA [98], FACTKG [101], WebQSP [209], GrailQA [64],
Graph Tasks	Accuracy, F1 Score, MRR, NDCG, Hit Ratio, MAE, RMSE	RelBench [156], TabGraphs [13], HiTab [34], TUDataset [135], GLBench [111], GPT4Graph [66], CS-TAG [199], GraphEval2000 [194]
Domain-specific Tasks	Accuracy, F1 Score, MRR, ROUGE, RMSD, Cumulative Return	STARK [196], AlphaFin [109], MedQA [95], BioRED [125], QALD [149], Mintaka [162], TutorQA [203], UltraDomain [153], PubMedQA [96]

Code Completion. In code completion tasks, using a graph with the RAG system helps capture and represent the complex relationships between code elements, such as control flow, data dependencies, and function calls. For example, GraphCoder [120] is an RAG framework designed to improve code completion by integrating repository-specific knowledge into code LLMs. Unlike traditional methods, it uses a **Code Context Graph** (**CCG**), which captures relationships like control flow and data dependence. The framework employs a two-step retrieval process: coarse-grained filtering to find candidate code snippets, followed by fine-grained re-ranking to prioritize those with aligned dependencies. This combination of structural and lexical context leads to more relevant code snippets, improving the accuracy of code generation. GraphCoder outperforms baseline methods, achieving higher code and identifier exact matches with lower time and storage costs. Its language-agnostic design, tested on Python and Java, proves its versatility across different programming environments.

Biomedical Question Answering. Biomedical knowledge is specialized and constantly changing, making it hard for traditional language models to provide accurate, up-to-date answers without access to external, structured information. GLBK [41] aims at tackling the information overload problem in biomedical question answering. The primary objective is to retrieve and prioritize relevant biomedical documents from a vast corpus, such as PubMed, while addressing complex, open-ended queries like identifying drug targets for diseases. Unlike traditional methods relying solely on embedding similarity, this approach integrates knowledge graph structures to mitigate biases from overrepresented topics and improve access to the long tail of biomedical knowledge. By leveraging entity recognition, relationship extraction, and graph-based rebalancing, the method enhances retrieval precision and ensures more contextually relevant information is surfaced.

7.4 Evaluation of Tasks

Open-source benchmark datasets. Following the taxonomy above, we also group the open-source benchmark datasets into three representative families (summarized in Table 6). For *KGQA*, WebQ [15] and WebQSP [209] focus on entity linking and compositional queries over knowledge graphs, while HotpotQA [205] emphasizes multi-hop aggregation and GrailQA [64] targets schema generalization across unseen relations. Recent temporal KGQA corpora—CronQuestion [160], Forecast [46], and MultiTQ [31]—introduce explicit time constraints to test era-aware retrieval and timestamp-valid answers. For *graph-centric tasks*, RelBench [156] provides standardized node, edge, and graph prediction suites. TUDataset [135] offers diverse graph classification collections, and GLBench [111] together with GraphEval2000 [194] concentrate on LLM-oriented graph reasoning and program execution. In *domain-specific settings*, MedQA [95] and PubMedQA [96] address clinically grounded QA with domain terminology and long-context evidence. BioRED [125] targets biomedical entity and relation extraction, and AlphaFin [109] represents financial analysis and forecasting under realistic market dynamics.

Evaluation metrics. We summarize widely used metrics across the three task families and note that Accuracy and F1 score are ubiquitous core measures for them. For answer quality in, **Exact Match** (**EM**), token-level F1 score, and Accuracy are standard, with **Recall-Oriented Understudy for Gisting Evaluation** (**ROUGE**), **Bilingual Evaluation Understudy** (**BLEU**), and **Metric for Evaluation of Translation with Explicit ORdering** (**METEOR**) applied when systems produce long-form justifications. For ranking and retrieval in graph reasoning, **Mean Reciprocal Rank** (**MRR**), **Normalized Discounted Cumulative Gain** (**NDCG**), and Hits@k are additionally reported. In domain-specific tasks, biomedical information extraction typically uses Accuracy and micro or macro F1; biology-oriented structure comparison relies on **Root Mean Square Deviation** (**RMSD**); and finance-oriented studies commonly track Cumulative Return, Sharpe Ratio, Maximum Drawdown and so on. Efficiency and stability are often captured with latency measures such as **Time to First Token** (**TTFT**) and robustness measures such as **Self-Consistency Rate** (**SCR**). The complete introduction of each metric is detailed in the online appendix [1].

7.5 Importance of Graph in Different Task Families

We finally examine the advantages of graph-based RAG over non-graph retrieval across three task families. (a) For simple factoid KGQA, a non-graph retriever often suffices because the answer typically resides in a single passage with little need for disambiguation. For complex KGQA, graphs add clear value by grounding entities and relations, composing multi-hop paths before generation, enforcing type and temporal constraints, constraining search to relevant neighborhoods, and preserving provenance for verification. These capabilities are instantiated by the Prize Collecting Steiner Tree module in G-Retriever, which selects an evidence subgraph aligned to the question, and the Personalized PageRank diffusion module in HippoRAG, which prioritizes neighborhoods seeded by detected entities to guide ranking and multi-hop reasoning. (b) For graph-centric tasks such as node classification and link prediction, performance hinges on local topology, relation types, and neighborhood signals. Graph-based RAG contributes by retrieving a compact subgraph anchored on the target nodes or node pairs, preserving adjacency and type constraints so the generator reasons over dense, topology-aware evidence within a small token budget. This reduces distraction from long passages, improves label and edge consistency, and supports clear provenance. These advantages are exemplified by subgraph message passing modules in RAGRAPH, which strengthen center node representations for node and link prediction, and the feature&text level enhancement modules in LLMs- EP, which inject two-hop structural signals to improve node classification. (c) Domain-specific tasks demand high accuracy and transparent explanations. Graph-based RAG strengthens accuracy by supporting loop and tree pipelines that let the model revisit, branch, and refine retrieval and generation across multiple passes. It also advances explainability by representing structured knowledge beyond classic knowledge graphs, including document, citation, clinical, and code graphs, so the system returns subgraphs with explicit entities, relations, and paths as verifiable evidence. These advantages are illustrated by the traversal agent in KGP, which iteratively expands and consolidates evidence on literature graphs, and the shortest-path retrieval in GLBK, which surfaces indirect biomedical relations with clear provenance.

8 Practical Applications in Industry

Industrial applications of graph-based RAG can be grouped into two tiers: production RAG systems and the workflow–platform stacks. The production tier includes Microsoft's GraphRAG,[12] which builds entity-based knowledge graphs and pre-generated community summaries to improve

[12] https://github.com/microsoft/graphrag

query-focused summarization; NebulaGraph's GraphRAG,[13] which integrates large-language-model reasoning with the NebulaGraph database; Eosphoros's GraphRAG,[14] which combines DB-GPT [227], OpenSPG,[15] and TuGraph [16] for triple extraction, subgraph location, and traversal; Neo4j's NaLLM,[17] which enables natural-language interfaces to knowledge graphs, construction from unstructured data, and report generation; Writer,[18] which utilizes graph-based RAG to enhance enterprise content generation and Q&A systems; and Leroy Merlin,[19] which uses product knowledge graphs to refine recommendation systems, providing personalized customer experiences. The workflow–platform tier comprises LangChain[20] and LangGraph[21] for graph-structured pipelines; LlamaIndex's Agentic Document Workflows[22] for end-to-end document processing and meta-agent coordination; multi-agent frameworks such as CrewAI[23] and AG2 AutoGen;[24] and cloud platforms including Vertex AI,[25] Amazon Bedrock,[26] and IBM watsonx.[27]

The Usage of Graph-based RAG in Frontier LLMs. Frontier LLMs such as GPT-5.2 [28] and Gemini 3 Pro [29] make graph-based RAG more practical because they are stronger at long-context understanding, tool selection, and multi-step planning. Therefore, they can reliably act as controllers and verifiers in retrieval pipelines. In particular, both ecosystems now provide effective retrieval tools named as File Search, which ingest enterprise documents, build searchable indexes, and retrieve only the most relevant evidence for generation.[30,31] Under our broad definition, this enables graph-based RAG beyond classic knowledge-graph traversal, where document-level structures and hierarchical graphs can be distilled from text and metadata, and retrieval can return compact subgraphs that enforce entity constraints before generation while reducing token and latency costs. Moreover, Gemini offers explicit controls for extended reasoning, enabling loop and tree style pipelines that iteratively retrieve evidence, verify intermediate results, and revise outputs, which matches the needs of graph-structured pipeline execution in practice.[32] Compared with earlier LLMs, frontier models mainly address two practical bottlenecks in RAG: they reduce *controller brittleness* for iterative retrieve–verify–revise pipelines, and they make RAG *scalable* to large private corpora by relying on managed retrieval interfaces instead of manual prompt stuffing [108].

Efficiency Discussion. The efficiency of industrial graph-based RAG can be examined through three views. First, graph construction and precomputation determine the baseline latency and storage footprint: Microsoft's GraphRAG constructs entity-centric knowledge graphs and pre-generates community summaries to speed query-focused summarization at the expense of build

[13] https://www.nebula-graph.io/posts/graph-RAG
[14] https://github.com/eosphoros-ai/DB-GPT
[15] https://github.com/OpenSPG/openspg
[16] https://tugraph.tech/
[17] https://github.com/neo4j/NaLLM
[18] https://writer.com/
[19] https://www.lettria.com/case-study/leroy-merlin-knowledge-graph-product-recommendations
[20] https://www.langchain.com/
[21] https://langchain-ai.github.io/langgraph
[22] https://www.llamaindex.ai/blog/agentic-rag-with-llamaindex-2721b8a49ff6
[23] https://github.com/crewAIInc/crewAI
[24] https://github.com/ag2ai/ag2
[25] https://cloud.google.com/vertex-ai
[26] https://aws.amazon.com/cn/bedrock/
[27] https://www.ibm.com/products/watsonx
[28] https://openai.com/index/introducing-gpt-5-2/
[29] https://ai.google.dev/gemini-api/docs/gemini-3
[30] https://platform.openai.com/docs/guides/migrate-to-responses
[31] https://ai.google.dev/gemini-api/docs/file-search
[32] https://ai.google.dev/gemini-api/docs/thinking

time and storage, while Neo4j's NaLLM automates the extraction of nodes, relations, and properties from unstructured data to reduce manual engineering effort in the indexing stage. Second, the retrieval strategy will control most online computation: AntGroup's GraphRAG narrows work by locating keywords, mapping them to graph nodes, and traversing only the necessary subgraph with breadth-first or depth-first search, and NebulaGraph's GraphRAG integrates large language models with the graph database to raise precision and avoid wasteful candidate expansion. Third, pipeline design governs coordination overhead and response quality. Sequential chains centralize control and keep latency low, exemplified by LangChain's modular RAG chains and CrewAI's support for sequential processes. Loop pipelines add iterative refinement and self-correction when increasing the cost. For example, LangGraph provides explicit loop capability with state persistence so a retriever and generator can revisit intermediate results when needed. Tree pipelines branch sub-queries or specialized agents for parallel search, which raises coordination cost but can improve answer quality. For instance, AG2 AutoGen together and OpenAI Swarm framework [33] fans out sub-queries before synthesis, which improves coverage at the cost of additional coordination.

9 Future Research Opportunities

Adaptive Prompts for LLMs. Future research could delve into the design and evaluation of adaptive prompts to better align with the pretrained knowledge of LLMs. By tailoring the structure, terminology, and hierarchy of information drawn from graphs, system designers could harness improved synergy between graph-based data and the LLM's reasoning capabilities. One could also investigate methods to prioritize the most critical entities, relationships, or paths in a query [113, 202], ensuring that the prompt remains precise and contextually rich without overwhelming the model. Such selective attention could enhance both the relevance and interpretability of LLM outputs. Furthermore, exploring feedback mechanisms that dynamically refine the prompt based on the LLM's responses might be an interesting direction, allowing systems to continuously optimize for coherence and clarity. Ultimately, adaptive prompt strategies could pave the way for more accurate, context-aware question answering, harnessing the potential of complex graph structures in tandem with cutting-edge language models.

Enhanced Understanding for Graph Problems. Future research could delve into approaches that enable deeper structural comprehension of graphs within LLMs. Since many current models rely on sequential token representations, they struggle to handle larger or more complex graph structures—especially when a sequence involves a large number of nodes [27, 66]. This limitation hinders their ability to solve typical graph-centric problems, such as path finding or community detection, which often require more specialized, non-linear reasoning. One direction might be to integrate graph-specific modules or adopt novel representations that faithfully capture relational information without overwhelming the model. Moreover, another avenue of exploration could involve combining LLMs with algorithmic components adopted at handling extensive, node-rich graphs.

Multi-Modal Graphs for Cross-Domain RAG. Future advancements in Multi-Modal Graphs for Cross-Domain RAG should focus on the development of adaptable retrieval algorithms that accommodate the diversity of data types encountered in real-world applications. Presently, graph-based RAG systems are largely confined to single-domain use cases, often employing retrieval methods tailored to specific types of textual information. However, as cross-domain scenarios become increasingly common, there is a pressing need for retrieval techniques that can seamlessly handle multi-modal graph structures, including images [71], audio [139], and numerical data. These enhanced retrieval algorithms must not only integrate and interpret a wide range of data formats but also dynamically adjust their retrieval strategies to match the unique characteristics of each domain.

[33] https://github.com/openai/swarm

By enabling a more flexible, multi-modal approach, future research in this field can overcome the limitations of current systems, paving the way for more robust and versatile knowledge acquisition across diverse domains.

Improved Graph Construction Techniques. The future of RAG systems can center around overcoming the limitations of traditional triple-based knowledge graphs, which are often too simplistic for complex, real-world data [104]. While the subject-predicate-object [214] format has served as a foundation, it does not fully capture intricate relationships and evolving patterns within knowledge. To address this, future graph construction methods should explore hypergraphs, which allow for multi-node relationships [106], and semantic embeddings [188] that can represent context, enhancing the depth and accuracy of graph structures. Additionally, GNNs can be used to learn graph representations that dynamically evolve as new information is introduced, allowing for more precise and adaptive knowledge storage. Another important direction will be the development of hierarchical graphs such as the approaches utilized in [210] that capture different levels of abstraction, providing a more flexible and structured way to organize knowledge. Furthermore, adaptive graph construction techniques, which adjust the structure and organization of the graph based on the type and volume of data, will be essential for handling real-time updates without sacrificing retrieval speed. In summary, these advanced techniques will allow RAG systems to store knowledge more efficiently, retrieve information faster, and support more complex, multi-modal data interactions. More future research opportunities can be found in our online appendix [1].

10 Conclusion

This article provides a comprehensive survey of the integration of graph-based techniques into RAG systems, offering a detailed review of their applications, advancements, and challenges. By categorizing existing methods and proposing a novel taxonomy, it provides valuable insights into how graphs can enhance the reasoning capabilities and accuracy of LLMs. The survey also identifies key challenges, such as dynamic graph integration, scalability, and explainability, while outlining future research directions to address these issues. Overall, this work contributes to a deeper understanding of the role of graphs in RAG systems and lays a foundation for future research aimed at unlocking their potential in improving LLM performance across a wide range of tasks.

References

[1] 2026. Retrieved from https://sites.google.com/view/ragsurvey

[2] Garima Agrawal, Tharindu Kumarage, Zeyad Alghamdi, and Huan Liu. 2023. Can knowledge graphs reduce hallucinations in llms?: A survey. *arXiv preprint arXiv:2311.07914* (2023). Retrieved from https://arxiv.org/abs/2311.07914

[3] Ahmet Yusuf Alan, Enis Karaarslan, and Omer Aydin. 2024. A RAG-based question answering system proposal for understanding islam: MufassirQAS LLM. *arXiv preprint arXiv:2401.15378* (2024). Retrieved from https://arxiv.org/abs/2401.15378

[4] Mohannad Alhanahnah, Yazan Boshmaf, and Benoit Baudry. 2024. DepsRAG: Towards Managing Software Dependencies using Large Language Models. arXiv:2405.20455. Retrieved from https://arxiv.org/abs/2405.20455

[5] Zhiyu An, Xianzhong Ding, Yen-Chun Fu, Cheng-Chung Chu, Yan Li, and Wan Du. 2024. Golden-Retriever: High-Fidelity Agentic Retrieval Augmented Generation for Industrial Knowledge Base. arXiv:2408.00798. Retrieved from https://arxiv.org/abs/2408.00798

[6] Gabor Angeli, Melvin Jose Johnson Premkumar, and Christopher D. Manning. 2015. Leveraging linguistic structure for open domain information extraction. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. The Association for Computer Linguistics, 344–354.

[7] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-rag: Learning to retrieve, generate, and critique through self-reflection. (2024).

[8] Sören Auer, Christian Bizer, Georgi Kobilarov, Jens Lehmann, Richard Cyganiak, and Zachary Ives. 2007. Dbpedia: A nucleus for a web of open data. In *Proceedings of the International Semantic Web Conference (ISWC/ASWC'07). (Lecture Notes in Computer Science, Vol. 4825)*. Springer, 722–735.

[9] Jinheon Baek, Alham Fikri Aji, Jens Lehmann, and Sung Ju Hwang. 2023. Direct fact retrieval from knowledge graphs without entity linking. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 10038–10055.

[10] Jinheon Baek, Alham Fikri Aji, and Amir Saffari. 2023. Knowledge-Augmented Language Model Prompting for Zero-Shot Knowledge Graph Question Answering. arXiv:2306.04136. Retrieved from https://arxiv.org/abs/2306.04136

[11] Jinheon Baek, Soyeong Jeong, Minki Kang, Jong C. Park, and Sung Ju Hwang. 2023. Knowledge-augmented language model verification. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 1720–1736.

[12] Ivana Balazevic, Carl Allen, and Timothy Hospedales. 2019. Multi-relational poincaré graph embeddings. *Advances in Neural Information Processing Systems* 32 (2019).

[13] Gleb Bazhenov, Oleg Platonov, and Liudmila Prokhorenkova. 2024. TabGraphs: A benchmark and strong baselines for learning on graphs with tabular node features. *arXiv preprint arXiv:2409.14500* (2024). Retrieved from https://arxiv.org/abs/2409.14500

[14] Richard Bellman. 1958. On a routing problem. *Quarterly of Applied Mathematics* 16, 1 (1958), 87–90.

[15] Jonathan Berant, Andrew Chou, Roy Frostig, and Percy Liang. 2013. Semantic parsing on freebase from question-answer pairs. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*. ACL, 1533–1544.

[16] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. 2024. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*. AAAI Press, 17682–17690.

[17] Maciej Besta, Florim Memedi, Zhenyu Zhang, Robert Gerstenberger, Nils Blach, Piotr Nyczyk, Marcin Copik, Grzegorz Kwasniewski, Jürgen Müller, Lukas Gianinazzi, et al. 2024. Topologies of reasoning: Demystifying chains, trees, and graphs of thoughts. arXiv:2401.14295. Retrieved from https://arxiv.org/abs/2401.14295

[18] Daniel Bienstock, Michel X. Goemans, David Simchi-Levi, and David Williamson. 1993. A note on the prize collecting traveling salesman problem. *Mathematical Programming* 59, 1 (1993), 413–420.

[19] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. 2020. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*. AAAI Press, 7432–7439.

[20] Kurt Bollacker, Colin Evans, Praveen Paritosh, Tim Sturge, and Jamie Taylor. 2008. Freebase: A collaboratively created graph database for structuring human knowledge. In *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*. 1247–1250.

[21] Antoine Bordes, Nicolas Usunier, Sumit Chopra, and Jason Weston. 2015. Large-scale simple question answering with memory networks. arXiv:1506.02075. Retrieved from https://arxiv.org/abs/1506.02075

[22] Astoria-Megler Bridge. 2001. Wikipedia, the free encyclopedia. *San Francisco (CA): Wikimedia Foundation* (2001).

[23] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in Neural Information Processing Systems* 33 (2020), 1877–1901.

[24] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, et al. 2023. Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv:2303.12712* (2023). Retrieved from https://arxiv.org/abs/2303.12712

[25] Abir Chakraborty. 2024. Multi-hop Question Answering over Knowledge Graphs using Large Language Models. arXiv:2404.19234. Retrieved from https://arxiv.org/abs/2404.19234

[26] Huajun Chen. 2024. Large Knowledge Model: Perspectives and Challenges. arXiv:2312.02706. Retrieved from https://arxiv.org/abs/2312.02706

[27] Nuo Chen, Yuhan Li, Jianheng Tang, and Jia Li. 2024. GraphWiz: An instruction-following language model for graph problems. *arXiv preprint arXiv:2402.16029* (2024). Retrieved from https://arxiv.org/abs/2402.16029

[28] Runjin Chen, Tong Zhao, Ajay Jaiswal, Neil Shah, and Zhangyang Wang. 2024. LLaGA: Large Language and Graph Assistant. arXiv:2402.08170.Retrieved from https://arxiv.org/abs/2402.08170

[29] Shuang Chen, Qian Liu, Zhiwei Yu, Chin-Yew Lin, Jian-Guang Lou, and Feng Jiang. 2021. ReTraCk: A flexible and efficient framework for knowledge base question answering. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 325–336.

[30] Yiqun Chen, Lingyong Yan, Weiwei Sun, Xinyu Ma, Yi Zhang, Shuaiqiang Wang, Dawei Yin, Yiming Yang, and Jiaxin Mao. 2025. Improving retrieval-augmented generation through multi-agent reinforcement learning. arXiv:1409.0473. Retrieved from https://arxiv.org/abs/1701.00133

[31] Ziyang Chen, Jinzhi Liao, and Xiang Zhao. 2023. Multi-granularity temporal question answering over knowledge graphs. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 11378–11392.

[32] Zhikai Chen, Haitao Mao, Hang Li, Wei Jin, Hongzhi Wen, Xiaochi Wei, Shuaiqiang Wang, Dawei Yin, Wenqi Fan, Hui Liu, et al. 2024. Exploring the potential of large language models (llms) in learning on graphs. *ACM SIGKDD Explorations Newsletter* 25, 2 (2024), 42–61.

[33] Keyuan Cheng, Gang Lin, Haoyang Fei, Yuxuan zhai, Lu Yu, Muhammad Asif Ali, Lijie Hu, and Di Wang. 2024. Multi-hop Question Answering under Temporal Knowledge Editing. arXiv:2404.00492. Retrieved from https://arxiv.org/abs/2404.00492

[34] Zhoujun Cheng, Haoyu Dong, Zhiruo Wang, Ran Jia, Jiaqi Guo, Yan Gao, Shi Han, Jian-Guang Lou, and Dongmei Zhang. 2021. Hitab: A hierarchical table dataset for question answering and natural language generation. *arXiv preprint arXiv:2108.06712* (2021). Retrieved from https://arxiv.org/abs/2108.06712

[35] Hyeong Kyu Choi, Seunghun Lee, Jaewon Chu, and Hyunwoo J. Kim. 2023. NuTrea: Neural Tree Search For Context-Guided Multi-hop KGQA. In *Proceedings of the NeurIPS*.

[36] Nurendra Choudhary and Chandan K. Reddy. 2024. Complex Logical Reasoning over Knowledge Graphs using Large Language Models. arXiv:2305.01157.Retrieved from https://arxiv.org/abs/2305.01157

[37] Rajarshi Das, Shehzaad Dhuliawala, Manzil Zaheer, Luke Vilnis, Ishan Durugkar, Akshay Krishnamurthy, Alex Smola, and Andrew McCallum. 2018. Go for a Walk and Arrive at the Answer: Reasoning Over Paths in Knowledge Bases using Reinforcement Learning. In *ICLR (Poster)*. OpenReview.net.

[38] DeepSeek-AI, Daya Guo, and et al. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948. Retrieved from https://arxiv.org/abs/2501.12948

[39] Mohammad Dehghan, Mohammad Ali Alomrani, Sunyam Bagga, David Alfonso-Hermelo, Khalil Bibi, Abbas Ghaddar, Yingxue Zhang, Xiaoguang Li, Jianye Hao, Qun Liu, et al. 2024. EWEK-QA : Enhanced Web and Efficient Knowledge Graph Retrieval for Citation-based Question Answering Systems. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 14169–14187.

[40] Yashar Deldjoo, Zhankui He, Julian McAuley, Anton Korikov, Scott Sanner, Arnau Ramisa, René Vidal, Maheswaran Sathiamoorthy, Atoosa Kasirzadeh, and Silvia Milano. 2024. A Review of Modern Recommender Systems Using Generative Models (Gen-RecSys). arXiv:2404.00579. Retrieved from https://arxiv.org/abs/2404.00579

[41] Julien Delile, Srayanta Mukherjee, Anton Van Pamel, and Leonid Zhukov. 2024. Graph-Based Retriever Captures the Long Tail of Biomedical Knowledge. arXiv:2402.12352. Retrieved from https://arxiv.org/abs/2402.12352

[42] Tim Dettmers, Pasquale Minervini, Pontus Stenetorp, and Sebastian Riedel. 2018. Convolutional 2D knowledge graph embeddings. In *Proceedings of the AAAI Conference on Artificial Intelligence*. AAAI Press, 1811–1818.

[43] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics, 4171–4186.

[44] Edsger W. Dijkstra. 2022. A note on two problems in connexion with graphs. In *Edsger Wybe Dijkstra: His Life, Work, and Legacy*. 287–290.

[45] Junhua Ding, Huyen Nguyen, and Haihua Chen. 2024. Evaluation of question-answering based text summarization using LLM invited paper. In *Proceedings of the 2024 IEEE International Conference on Artificial Intelligence Testing* . IEEE, 142–149.

[46] Zifeng Ding, Zongyue Li, Ruoxia Qi, Jingpei Wu, Bailan He, Yunpu Ma, Zhao Meng, Shuo Chen, Ruotong Liao, Zhen Han, et al. 2023. Forecasttkgquestions: A benchmark for temporal question answering and forecasting over temporal knowledge graphs. In *Proceedings of the International Semantic Web Conference*. Springer, 541–560.

[47] Guanting Dong, Rumei Li, Sirui Wang, Yupeng Zhang, Yunsen Xian, and Weiran Xu. 2023. Bridging the KB-text gap: Leveraging structured knowledge-aware pre-training for KBQA. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. ACM, 3854–3859.

[48] Junnan Dong, Qinggang Zhang, Xiao Huang, Keyu Duan, Qiaoyu Tan, and Zhimeng Jiang. 2023. Hierarchy-aware multi-hop question answering over knowledge graphs. In *Proceedings of the ACM Web Conference 2023, Austin, TX, USA, 30 April 2023 - 4 May 2023*. ACM, 2519–2527.

[49] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130. Retrieved from https://arxiv.org/abs/2404.16130

[50] Hady ElSahar, Pavlos Vougiouklis, Arslen Remaci, Christophe Gravier, Jonathon S. Hare, Frédérique Laforest, and Elena Simperl. 2018. T-REx: A large scale alignment of natural language with knowledge base triples. In *Proceedings of the 11th International Conference on Language Resources and Evaluation*. European Language Resources Association (ELRA).

[51] Oren Etzioni, Michele Banko, Stephen Soderland, and Daniel S. Weld. 2008. Open information extraction from the web. *Communications of the ACM* 51, 12 (2008), 68–74.

[52] Wenqi Fan, Yujuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. 2024. A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models. arXiv:2405.06211. Retrieved from https://arxiv.org/abs/2405.06211

[53] Wenqi Fan, Shijie Wang, Jiani Huang, Zhikai Chen, Yu Song, Wenzhuo Tang, Haitao Mao, Hui Liu, Xiaorui Liu, Dawei Yin, et al. 2024. Graph Machine Learning in the Era of Large Language Models (LLMs). arXiv:2404.14928. Retrieved from https://arxiv.org/abs/2404.14928

[54] Jinyuan Fang, Zaiqiao Meng, and Craig MacDonald. 2024. REANO: Optimising retrieval-augmented reader models through knowledge graph generation. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2094–2112.

[55] Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. 2023. Talk like a Graph: Encoding Graphs for Large Language Models. arXiv:2310.04560. Retrieved from https://arxiv.org/abs/2310.04560

[56] Chao Feng, Xinyu Zhang, and Zichu Fei. 2023. Knowledge Solver: Teaching LLMs to Search for Domain Knowledge from Knowledge Graphs. arXiv:2309.03118. Retrieved from https://arxiv.org/abs/2309.03118

[57] Yanlin Feng, Xinyue Chen, Bill Yuchen Lin, Peifeng Wang, Jun Yan, and Xiang Ren. 2020. Scalable multi-hop relational reasoning for knowledge-aware question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 1295–1309.

[58] Zhaopeng Feng, Yan Zhang, Hao Li, Wenqiang Liu, Jun Lang, Yang Feng, Jian Wu, and Zuozhu Liu. 2024. Improving llm-based machine translation with systematic self-correction. *arXiv preprint arXiv:2402.16379* (2024). Retrieved from https://arxiv.org/abs/2402.16379

[59] Paolo Ferragina and Ugo Scaiella. 2010. Tagme: On-the-fly annotation of short text fragments (by wikipedia entities). In *Proceedings of the 19th ACM International Conference on Information and Knowledge Management*. 1625–1628.

[60] D. Flanagan. 2007. Developing metaweb-enabled web applications. *Metaweb Technologies* (2007).

[61] Hanning Gao, Lingfei Wu, Po Hu, Zhihua Wei, Fangli Xu, and Bo Long. 2022. Graph-augmented learning to rank for querying large-scale knowledge graph. In *Proceedings of the 2nd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics and the 12th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Association for Computational Linguistics, 82–92.

[62] Yifu Gao, Linbo Qiao, Zhigang Kan, Zhihua Wen, Yongquan He, and Dongsheng Li. 2024. Two-stage generative question answering on temporal knowledge graph using large language models. arXiv:2402.16568. Retrieved from https://arxiv.org/abs/2402.16568

[63] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2024. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997. Retrieved from https://arxiv.org/abs/2312.10997

[64] Yu Gu, Sue Kase, Michelle Vanni, Brian M. Sadler, Percy Liang, Xifeng Yan, and Yu Su. 2021. Beyond I.I.D.: Three levels of generalization for question answering on knowledge bases. In *Proceedings of the Web Conference 2021, Virtual Event / Ljubljana, Slovenia, April 19-23, 2021*. 3477–3488.

[65] Yu Gu and Yu Su. 2022. ArcaneQA: Dynamic program induction and contextualized encoding for knowledge base question answering. In *Proceedings of the 29th International Conference on Computational Linguistics*. 1718–1731.

[66] Jiayan Guo, Lun Du, Hengyu Liu, Mengyu Zhou, Xinyi He, and Shi Han. 2023. GPT4Graph: Can Large Language Models Understand Graph Structured Data ? An Empirical Evaluation and Benchmarking. arXiv:2305.15066. Retrieved from https://arxiv.org/abs/2305.15066

[67] Tiezheng Guo, Qingwen Yang, Chen Wang, Yanyi Liu, Pan Li, Jiawei Tang, Dapeng Li, and Yingyou Wen. 2024. KnowledgeNavigator: Leveraging Large Language Models for Enhanced Reasoning over Knowledge Graph. arXiv:2312.15880. Retrieved from https://arxiv.org/abs/2312.15880

[68] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. arXiv:2405.14831. Retrieved from https://arxiv.org/abs/2405.14831

[69] Muhammad Usman Hadi, Rizwan Qureshi, Abbas Shah, Muhammad Irfan, Anas Zafar, Muhammad Bilal Shaikh, Naveed Akhtar, Jia Wu, Seyedali Mirjalili, et al. 2023. A survey on large language models: Applications, challenges, limitations, and practical usage. *Authorea Preprints* (2023).

[70] Haoyu Han, Yu Wang, Harry Shomer, Kai Guo, Jiayuan Ding, Yongjia Lei, Mahantesh Halappanavar, Ryan A Rossi, Subhabrata Mukherjee, Xianfeng Tang, et al. 2024. Retrieval-augmented generation with graphs (graphrag). *arXiv preprint arXiv:2501.00309* (2024). Retrieved from https://arxiv.org/abs/2501.00309

[71] Kai Han, Yunhe Wang, Jianyuan Guo, Yehui Tang, and Enhua Wu. 2022. Vision gnn: An image is worth graph of nodes. *Advances in Neural Information Processing Systems* 35 (2022), 8291–8303.

[72] Zhen Han, Yue Feng, and Mingming Sun. 2023. A graph-guided reasoning approach for open-ended commonsense question answering. arXiv:2303.10395. Retrieved from https://arxiv.org/abs/2303.10395

[73] Gaole He, Yunshi Lan, Jing Jiang, Wayne Xin Zhao, and Ji-Rong Wen. 2021. Improving multi-hop knowledge base question answering by learning intermediate supervision signals. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*. ACM, 553–561.

[74] Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh V. Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. 2024. G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering. arXiv:2402.07630. Retrieved from https://arxiv.org/abs/2402.07630

[75] Johannes Hoffart, Mohamed Amir Yosef, Ilaria Bordino, Hagen Fürstenau, Manfred Pinkal, Marc Spaniol, Bilyana Taneva, Stefan Thater, and Gerhard Weikum. 2011. Robust disambiguation of named entities in text. In *Proceedings of the 2011 Conference on Empirical Methods in Natural Language Processing*. ACL, 782–792.

[76] Yuntong Hu, Zhihan Lei, Zheng Zhang, Bo Pan, Chen Ling, and Liang Zhao. 2024. GRAG: Graph Retrieval-Augmented Generation. arXiv:2405.16506. Retrieved from https://arxiv.org/abs/2405.16506

[77] Yucheng Hu and Yuxing Lu. 2024. RAG and RAU: A Survey on Retrieval-Augmented Language Model in Natural Language Processing. arXiv:2404.19543. Retrieved from https://arxiv.org/abs/2404.19543

[78] Ziniu Hu, Yichong Xu, Wenhao Yu, Shuohang Wang, Ziyi Yang, Chenguang Zhu, Kai-Wei Chang, and Yizhou Sun. 2022. Empowering language models with knowledge graph reasoning for open-domain question answering. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 9562–9581.

[79] Jie Huang and Kevin Chen-Chuan Chang. 2022. Towards reasoning in large language models: A survey. arXiv:2212.10403. Retrieved from https://arxiv.org/abs/2212.10403

[80] Jerry Huang, Siddarth Madala, Risham Sidhu, Cheng Niu, Hao Peng, Julia Hockenmaier, and Tong Zhang. 2025. Rag-rl: Advancing retrieval-augmented generation via rl and curriculum learning. *arXiv preprint arXiv:2503.12759* (2025). Retrieved from https://arxiv.org/abs/2503.12759

[81] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, and Bing Qin. 2023. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. arXiv:2311.05232. Retrieved from https://arxiv.org/abs/2311.05232

[82] Yizheng Huang and Jimmy Huang. 2024. A Survey on Retrieval-Augmented Text Generation for Large Language Models. arXiv:2404.10981.Retrieved from https://arxiv.org/abs/2404.10981

[83] Yongfeng Huang, Yanyang Li, Yichong Xu, Lin Zhang, Ruyi Gan, Jiaxing Zhang, and Liwei Wang. 2023. MVP-tuning: Multi-view knowledge retrieval with prompt tuning for commonsense reasoning. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 13417–13432.

[84] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. 2024. Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity. arXiv:2403.14403. Retrieved from https://arxiv.org/abs/2403.14403

[85] Jinhao Jiang, Kun Zhou, Zican Dong, Keming Ye, Xin Zhao, and Ji-Rong Wen. 2023. StructGPT: A general framework for large language model to reason over structured data. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 9237–9251.

[86] Jinhao Jiang, Kun Zhou, Wayne Xin Zhao, Yang Song, Chen Zhu, Hengshu Zhu, and Ji-Rong Wen. 2024. KG-Agent: An Efficient Autonomous Agent Framework for Complex Reasoning over Knowledge Graph. arXiv:2402.11163. Retrieved from https://arxiv.org/abs/2402.11163

[87] Jinhao Jiang, Kun Zhou, Xin Zhao, and Ji-Rong Wen. 2023. UniKGQA: Unified retrieval and reasoning for solving multi-hop question answering over knowledge graph. In *Proceedings of the 11th International Conference on Learning Representations ICLR 2023*. Kigali, Rwanda.

[88] Kelvin Jiang, Dekun Wu, and Hui Jiang. 2019. FreebaseQA: A new factoid QA data set matching trivia-style question-answer pairs with freebase. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics, 318–323.

[89] Xinke Jiang, Rihong Qiu, Yongxin Xu, Wentao Zhang, Yichen Zhu, Ruizhe Zhang, Yuchen Fang, Xu Chu, Junfeng Zhao, and Yasha Wang. 2024. RAGraph: A general retrieval-augmented graph learning framework. arXiv:2410.23855. Retrieved from https://arxiv.org/abs/2410.23855

[90] Xinke Jiang, Ruizhe Zhang, Yongxin Xu, Rihong Qiu, Yue Fang, Zhiyuan Wang, Jinyi Tang, Hongxin Ding, Xu Chu, Junfeng Zhao, and Yasha Wang. 2024. HyKGE: A Hypothesis Knowledge Graph Enhanced Framework for Accurate and Reliable Medical LLMs Responses. arXiv:2312.15883. Retrieved from https://arxiv.org/abs/2312.15883

[91] Yikun Jiang, Huanyu Wang, Lei Xie, Hanbin Zhao, Hui Qian, John Lui, et al. 2025. D-llm: A token adaptive computing resource allocation strategy for large language models. *Advances in Neural Information Processing Systems* 37 (2025), 1725–1749.

[92] Zhengbao Jiang, Frank F Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 7969–7992.

[93] Bowen Jin, Gang Liu, Chi Han, Meng Jiang, Heng Ji, and Jiawei Han. 2024. Large Language Models on Graphs: A Comprehensive Survey. arXiv:2312.02783. Retrieved from https://arxiv.org/abs/2312.02783

[94] Bowen Jin, Chulin Xie, Jiawei Zhang, Kashob Kumar Roy, Yu Zhang, Zheng Li, Ruirui Li, Xianfeng Tang, Suhang Wang, Yu Meng, and Jiawei Han. 2024. Graph chain-of-thought: Augmenting large language models by reasoning on graphs. arXiv:2404.07103. Retrieved from https://arxiv.org/abs/2404.07103

[95] Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. 2020. What Disease does this Patient Have? A Large-scale Open Domain Question Answering Dataset from Medical Exams. arXiv:2009.13081. Retrieved from https://arxiv.org/abs/2009.13081

[96] Qiao Jin, Bhuwan Dhingra, Zhengping Liu, William W. Cohen, and Xinghua Lu. 2019. Pubmedqa: A dataset for biomedical research question answering. *arXiv preprint arXiv:1909.06146* (2019). Retrieved from https://arxiv.org/abs/1909.06146

[97] Jaehyeong Jo, Jinheon Baek, Seul Lee, Dongki Kim, Minki Kang, and Sung Ju Hwang. 2021. Edge representation learning with hypergraphs. *Advances in Neural Information Processing Systems* 34 (2021), 7534–7546.

[98] Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. 2017. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics, ACL 2017, Vancouver, Canada, July 30 - August 4, Volume 1: Long Papers*. 1601–1611.

[99] Minki Kang, Jin Myung Kwak, Jinheon Baek, and Sung Ju Hwang. 2022. Knowledge-consistent dialogue generation with knowledge graphs. In *Proceedings of the ICML 2022 Workshop on Knowledge Retrieval and Language Models*.

[100] Jiho Kim, Yeonsu Kwon, Yohan Jo, and Edward Choi. 2023. KG-GPT: A general framework for reasoning on knowledge graphs using large language models. In *Findings of the Association for Computational Linguistics, Singapore, December 6-10, 2023*. 9410–9421.

[101] Jiho Kim, Sungjin Park, Yeonsu Kwon, Yohan Jo, James Thorne, and Edward Choi. 2023. FactKG: Fact verification via reasoning on knowledge graphs. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2023, Toronto, Canada, July 9-14, 2023*. 16190–16206.

[102] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *Proceedings of the 5th International Conference on Learning Representations, Toulon, France, April 24-26, 2017, Conference Track Proceedings*.

[103] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur P. Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. 2019. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics* 7 (2019), 452–466.

[104] Yunshi Lan and Jing Jiang. 2020. Query graph generation for answering multi-hop complex questions from knowledge bases. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Online, July 5-10, 2020*. 969–974.

[105] Ernests Lavrinovics, Russa Biswas, Johannes Bjerva, and Katja Hose. 2025. Knowledge graphs, large language models, and hallucinations: An NLP perspective. *Journal of Web Semantics* 85 (2025), 100844.

[106] Ai Li and Steve Horvath. 2007. Network neighborhood analysis with the multi-node topological overlap measure. *Bioinformatics* 23, 2 (2007), 222–231.

[107] Dawei Li, Shu Yang, Zhen Tan, Jae Young Baik, Sukwon Yun, Joseph Lee, Aaron Chacko, Bojian Hou, Duy Duong-Tran, Ying Ding, et al. 2024. DALK: Dynamic Co-Augmentation of LLMs and KG to answer Alzheimer's Disease Questions with Scientific Literature. arXiv:2405.04819. Retrieved from https://arxiv.org/abs/2405.04819

[108] Xinze Li, Yushi Bai, Bowen Jin, Fengbin Zhu, Liangming Pan, and Yixin Cao. 2025. Long context vs. RAG: Strategies for processing long documents in LLMs. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 4110–4113.

[109] Xiang Li, Zhenyu Li, Chen Shi, Yong Xu, Qing Du, Mingkui Tan, Jun Huang, and Wei Lin. 2024. AlphaFin: Benchmarking financial analysis with retrieval-augmented stock-chain framework. arXiv:2403.12582. Retrieved from https://arxiv.org/abs/2403.12582

[110] Yuhan Li, Zhixun Li, Peisong Wang, Jia Li, Xiangguo Sun, Hong Cheng, and Jeffrey Xu Yu. 2024. A Survey of Graph Meets Large Language Model: Progress and Future Directions. arXiv:2311.12399. Retrieved from https://arxiv.org/abs/2311.12399

[111] Yuhan Li, Peisong Wang, Xiao Zhu, Aochuan Chen, Haiyun Jiang, Deng Cai, Victor Wai Kin Chan, and Jia Li. 2024. GLBench: A comprehensive benchmark for graph with large language models. *ArXiv* abs/2407.07457 (2024). Retrived from https://api.semanticscholar.org/CorpusID:271088951

[112] Zhuoyang Li, Liran Deng, Hui Liu, Qiaoqiao Liu, and Junzhao Du. 2024. UniOQA: A Unified Framework for Knowledge Graph Question Answering with Large Language Models. arXiv:2406.02110. Retrieved from https://arxiv.org/abs/2406.02110

[113] Zijian Li, Qingyan Guo, Jiawei Shao, Lei Song, Jiang Bian, Jun Zhang, and Rui Wang. 2024. Graph Neural Network Enhanced Retrieval for Question Answering of LLMs. arXiv:2406.06572. Retrieved from https://arxiv.org/abs/2406.06572

[114] Bill Yuchen Lin, Xinyue Chen, Jamin Chen, and Xiang Ren. 2019. KagNet: Knowledge-aware graph networks for commonsense reasoning. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*. Association for Computational Linguistics, 2829–2839.

[115] Bill Yuchen Lin, Ziyi Wu, Yichi Yang, Dong-Ho Lee, and Xiang Ren. 2021. RiddleSense: Reasoning about riddle questions featuring linguistic creativity and commonsense knowledge. In *ACL/IJCNLP (Findings) (Findings of ACL, Vol. ACL/IJCNLP 2021)*. Association for Computational Linguistics, 1504–1515.

[116] Guangyi Liu, Yongqi Zhang, Yong Li, and Quanming Yao. 2024. Explore then Determine: A GNN-LLM Synergy Framework for Reasoning over Knowledge Graph. arXiv:2406.01145. Retrieved from https://arxiv.org/abs/2406.01145

[117] Haochen Liu, Song Wang, Yaochen Zhu, Yushun Dong, and Jundong Li. 2024. Knowledge graph-enhanced large language models via path selection. In *Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024*. 6311–6321.

[118] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2024. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems* 36 (2024).

[119] Jiawei Liu, Cheng Yang, Zhiyuan Lu, Junze Chen, Yibo Li, Mengmei Zhang, Ting Bai, Yuan Fang, Lichao Sun, Philip S. Yu, et al. 2024. Towards graph foundation models: A survey and beyond. arXiv:2310.11829. Retrieved from https://arxiv.org/abs/2310.11829

[120] Wei Liu, Ailun Yu, Daoguang Zan, Bo Shen, Wei Zhang, Haiyan Zhao, Zhi Jin, and Qianxiang Wang. 2024. GraphCoder: Enhancing repository-level code completion via code context graph-based retrieval and language model. *arXiv preprint arXiv:2406.07003* (2024). Retrieved from https://arxiv.org/abs/2406.07003

[121] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A robustly optimized BERT pretraining approach. arXiv:1907.11692. Retrieved from https://arxiv.org/abs/1907.11692

[122] Lajanugen Logeswaran, Ming-Wei Chang, Kenton Lee, Kristina Toutanova, Jacob Devlin, and Honglak Lee. 2019. Zero-shot entity linking by reading entity descriptions. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 3449–3460.

[123] Dan Luo, Jiawei Sheng, Hongbo Xu, Lihong Wang, and Bin Wang. 2023. Improving complex knowledge base question answering with relation-aware subgraph retrieval and reasoning network. In *Proceedings of the International Joint Conference on Neural Networks, Gold Coast, Australia, June 18-23, 2023*. 1–8.

[124] Haoran Luo, Haihong E, Zichen Tang, Shiyao Peng, Yikai Guo, Wentai Zhang, Chenghao Ma, Guanting Dong, Meina Song, Wei Lin, et al. 2024. ChatKBQA: A Generate-then-Retrieve Framework for Knowledge Base Question Answering with Fine-tuned Large Language Models. arXiv:2310.08975. Retrieved from https://arxiv.org/abs/2310.08975

[125] Ling Luo, Po-Ting Lai, Chih-Hsuan Wei, Cecilia N. Arighi, and Zhiyong Lu. 2022. BioRED: A rich biomedical relation extraction dataset. *Briefings Bioinform.* 23, 5 (2022). DOI: https://doi.org/10.1093/BIB/BBAC282

[126] Linhao Luo, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan. 2024. Reasoning on graphs: Faithful and interpretable large language model reasoning. arXiv:2310.01061. Retrieved from https://arxiv.org/abs/2310.01061

[127] Yuanjie Lyu, Zhiyu Li, Simin Niu, Feiyu Xiong, Bo Tang, Wenjin Wang, Hao Wu, Huanyong Liu, Tong Xu, and Enhong Chen. 2024. CRUD-RAG: A comprehensive chinese benchmark for retrieval-augmented generation of large language models. *arXiv preprint arXiv:2401.17043* (2024). Retrieved from https://arxiv.org/abs/2401.17043

[128] Shengjie Ma, Chengjin Xu, Xuhui Jiang, Muzhi Li, Huaren Qu, and Jian Guo. 2024. Think-on-graph 2.0: Deep and interpretable large language model reasoning with knowledge graph-guided retrieval. arXiv:2407.10805. Retrieved from https://arxiv.org/abs/2407.10805

[129] Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. 2023. Query rewriting for retrieval-augmented large language models. arXiv:2305.14283. Retrieved from https://arxiv.org/abs/2305.14283

[130] Qiheng Mao, Zemin Liu, Chenghao Liu, Zhuo Li, and Jianling Sun. 2024. Advancing graph representation learning with large language models: A comprehensive survey of techniques. arXiv:2402.05952. Retrieved from https://arxiv.org/abs/2402.05952

[131] Costas Mavromatis and George Karypis. 2024. GNN-RAG: Graph neural retrieval for large language model reasoning. arXiv:2405.20139. Retrieved from https://arxiv.org/abs/2405.20139

[132] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. 2018. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2381–2391.

[133] Shervin Minaee, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao. 2024. Large language models: A survey. arXiv:2402.06196. Retrieved from https://arxiv.org/abs/2402.06196

[134] Debjyoti Mondal, Suraj Modi, Subhadarshi Panda, Rituraj Singh, and Godawari Sudhakar Rao. 2024. KAM-CoT: Knowledge augmented multimodal chain-of-thoughts reasoning. In *Proceedings of the AAAI*. AAAI Press, 18798–18806.

[135] Christopher Morris, Nils M. Kriege, Franka Bause, Kristian Kersting, Petra Mutzel, and Marion Neumann. 2020. TU-Dataset: A collection of benchmark datasets for learning with graphs. In *ICML 2020 Workshop on Graph Representation Learning and Beyond (GRL+ 2020)*.

[136] Sai Munikoti, Anurag Acharya, Sridevi Wagle, and Sameera Horawalavithana. 2023. ATLANTIC: Structure-Aware Retrieval-Augmented Language Model for Interdisciplinary Science. arXiv:2311.12289. Retrieved from https://arxiv.org/abs/2311.12289

[137] Devon Myers, Rami Mohawesh, Venkata Ishwarya Chellaboina, Anantha Lakshmi Sathvik, Praveen Venkatesh, Yi-Hui Ho, Hanna Henshaw, Muna Alhawawreh, David Berdik, and Yaser Jararweh. 2024. Foundation and large language models: fundamentals, challenges, opportunities, and social impacts. *Cluster Computing* 27, 1 (2024), 1–26.

[138] Humza Naveed, Asad Ullah Khan, Shi Qiu, Muhammad Saqib, Saeed Anwar, Muhammad Usman, Naveed Akhtar, Nick Barnes, and Ajmal Mian. 2023. A comprehensive overview of large language models. arXiv:2307.06435. Retrieved from https://arxiv.org/abs/2307.06435

[139] Weizhi Nie, Minjie Ren, Jie Nie, and Sicheng Zhao. 2020. C-GCN: Correlation based graph convolutional network for audio-video emotion recognition. *IEEE Transactions on Multimedia* 23 (2020), 3793–3804.

[140] Armin Oliya, Amir Saffari, Priyanka Sen, and Tom Ayoola. 2021. End-to-end entity resolution and question answering using differentiable knowledge graphs. *arXiv preprint arXiv:2109.05817* (2021). Retrieved from https://arxiv.org/abs/2109.05817

[141] Yasumasa Onoe, Michael J. Q. Zhang, Eunsol Choi, and Greg Durrett. 2021. CREAK: A dataset for commonsense reasoning over entity knowledge. In *Proceedings of the NeurIPS Datasets and Benchmarks*.

[142] OpenAI. 2024. GPT-4 Technical Report. arXiv:2303.08774. Retrieved from https://arxiv.org/abs/2303.08774

[143] Tobias A. Opsahl. 2024. Fact or fiction? Improving fact verification with knowledge graphs through simplified subgraph retrievals. arXiv:2408.07453. Retrieved from https://arxiv.org/abs/2408.07453

[144] Vardaan Pahuja, Boshi Wang, Hugo Latapie, Jayanth Srinivasa, and Yu Su. 2023. A retrieve-and-read framework for knowledge graph link prediction. In *Proceedings of the CIKM*. ACM, 1992–2002.

[145] Jeff Z. Pan, Simon Razniewski, Jan-Christoph Kalo, Sneha Singhania, Jiaoyan Chen, Stefan Dietze, Hajira Jabeen, Janna Omeliyanenko, Wen Zhang, Matteo Lissandrini, et al. 2023. Large language models and knowledge graphs: Opportunities and challenges. *TGDK* 1, 1 (2023), 2:1–2:38.

[146] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. 2024. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering* 36, 7 (2024), 3580–3599.

[147] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. 2024. Graph retrieval-augmented generation: A survey. arXiv:2408.08921. Retrieved from https://arxiv.org/abs/2408.08921

[148] Zhuoyi Peng and Yi Yang. 2024. Connecting the Dots: Inferring patent phrase similarity with retrieved phrase graphs. arXiv:2403.16265. Retrieved from https://arxiv.org/abs/2403.16265

[149] Aleksandr Perevalov, Dennis Diefenbach, Ricardo Usbeck, and Andreas Both. 2022. QALD-9-plus: A multilingual dataset for question answering over DBpedia and Wikidata translated by native speakers. In *Proceedings of the ICSC*. IEEE, 229–234.

[150] Fabio Petroni, Aleksandra Piktus, Angela Fan, Patrick Lewis, Majid Yazdani, Nicola De Cao, James Thorne, Yacine Jernite, Vladimir Karpukhin, Jean Maillard, et al. 2021. KILT: A benchmark for knowledge intensive language tasks. In *Proceedings of the NAACL-HLT*. Association for Computational Linguistics, 2523–2544.

[151] Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. 2022. Measuring and narrowing the compositionality gap in language models. arXiv:2210.03350. Retrieved from https://arxiv.org/abs/2210.03350

[152] Zhixiao Qi, Yijiong Yu, Meiqi Tu, Junyi Tan, and Yongfeng Huang. 2023. FoodGPT: A large language model in food testing domain with incremental pre-training and knowledge graph prompt. arXiv:2308.10173. Retrieved from https://arxiv.org/abs/2308.10173

[153] Hongjin Qian, Peitian Zhang, Zheng Liu, Kelong Mao, and Zhicheng Dou. 2024. Memorag: Moving towards next-gen rag via memory-inspired knowledge discovery. *arXiv preprint arXiv:2409.05591*. 1 (2024).

[154] Priyanka Ranade and Anupam Joshi. 2023. FABULA: Intelligence report generation using retrieval-augmented narrative construction. In *Proceedings of the International Conference on Advances in Social Networks Analysis and Mining, Kusadasi, Turkey, November 6-9, 2023*. 603–610.

[155] Vipula Rawte, Amit Sheth, and Amitava Das. 2023. A survey of hallucination in large foundation models. arXiv:2309.05922. Retrieved from https://arxiv.org/abs/2309.05922

[156] Joshua Robinson, Rishabh Ranjan, Weihua Hu, Kexin Huang, Jiaqi Han, Alejandro Dobles, Matthias Fey, Jan Eric Lenssen, Yiwen Yuan, Zecheng Zhang, et al. 2024. Relbench: A benchmark for deep learning on relational databases. *Advances in Neural Information Processing Systems* 37 (2024), 21330–21341.

[157] Maarten Sap, Ronan Le Bras, Emily Allaway, Chandra Bhagavatula, Nicholas Lourie, Hannah Rashkin, Brendan Roof, Noah A. Smith, and Yejin Choi. 2019. ATOMIC: An atlas of machine commonsense for if-then reasoning. In *Proceedings of the AAAI*. AAAI Press, 3027–3035.

[158] Maarten Sap, Hannah Rashkin, Derek Chen, Ronan LeBras, and Yejin Choi. 2019. SocialIQA: Commonsense Reasoning about Social Interactions. arXiv:1904.09728. Retrieved from https://arxiv.org/abs/1904.09728

[159] Bhaskarjit Sarmah, Benika Hall, Rohan Rao, Sunil Patel, Stefano Pasquali, and Dhagash Mehta. 2024. HybridRAG: Integrating knowledge graphs and vector retrieval augmented generation for efficient information extraction. arXiv:2408.04948. Retrieved from https://arxiv.org/abs/2408.04948

[160] Apoorv Saxena, Soumen Chakrabarti, and Partha Talukdar. 2021. Question answering over temporal knowledge graphs. *arXiv preprint arXiv:2106.01515* (2021). Retrieved from https://arxiv.org/abs/2106.01515

[161] Edward R. Scheinerman and Daniel H. Ullman. 2011. *Fractional Graph Theory: A Rational Approach to the Theory of Graphs*. Courier Corporation.

[162] Priyanka Sen, Alham Fikri Aji, and Amir Saffari. 2022. Mintaka: A complex, natural, and multilingual dataset for end-to-end question answering. In *Proceedings of the 29th International Conference on Computational Linguistics, Gyeongju, Republic of Korea, October 12-17, 2022*. 1604–1619.

[163] Priyanka Sen, Sandeep Mavadia, and Amir Saffari. 2023. Knowledge graph-augmented language models for complex question answering. In *Proceedings of the 1st Workshop on Natural Language Reasoning and Structured Explanations* . 1–8.

[164] Tianhao Shen, Renren Jin, Yufei Huang, Chuang Liu, Weilong Dong, Zishan Guo, Xinwei Wu, Yan Liu, and Deyi Xiong. 2023. Large language model alignment: A survey. arXiv:2309.15025. Retrieved from https://arxiv.org/abs/2309.15025

[165] Yunsheng Shi, Zhengjie Huang, Shikun Feng, Hui Zhong, Wenjin Wang, and Yu Sun. 2020. Masked label prediction: Unified message passing model for semi-supervised classification. *arXiv preprint arXiv:2009.03509* (2020). Retrieved from https://arxiv.org/abs/2009.03509

[166] Kurt Shuster, Spencer Poff, Moya Chen, Douwe Kiela, and Jason Weston. 2021. Retrieval augmentation reduces hallucination in conversation. In *Proceedings of the EMNLP (Findings)*. Association for Computational Linguistics, 3784–3803.

[167] Fabian M. Suchanek, Gjergji Kasneci, and Gerhard Weikum. 2007. Yago: A core of semantic knowledge. In *Proceedings of the 16th International Conference on World Wide Web*. 697–706.

[168] Haitian Sun, Tania Bedrax-Weiss, and William W. Cohen. 2019. PullNet: Open domain question answering with iterative retrieval on knowledge bases and text. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, Hong Kong, China, November 3-7, 2019*. 2380–2390.

[169] Hao Sun, Yang Li, Liwei Deng, Bowen Li, Binyuan Hui, Binhua Li, Yunshi Lan, Yan Zhang, and Yongbin Li. 2023. History semantic graph enhanced conversational KBQA with temporal information modeling. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2023, Toronto, Canada, July 9-14, 2023*. 3521–3533.

[170] Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M. Ni, Heung-Yeung Shum, and Jian Guo. 2024. Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph. arXiv:2307.07697. Retrieved from https://arxiv.org/abs/2307.07697

[171] Lei Sun, Zhengwei Tao, Youdi Li, and Hiroshi Arakawa. 2024. ODA: Observation-driven agent for integrating LLMs and knowledge graphs. arXiv:2404.07677. Retrieved from https://arxiv.org/abs/2404.07677

[172] Alon Talmor and Jonathan Berant. 2018. The web as a knowledge-base for answering complex questions. In *Proceedings of the NAACL-HLT*. Association for Computational Linguistics, 641–651.

[173] Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. 2019. CommonsenseQA: A question answering challenge targeting commonsense knowledge. In *Proceedings of the NAACL-HLT (1)*. Association for Computational Linguistics, 4149–4158.

[174] Yanchao Tan, Hang Lv, Xinyi Huang, Jiawei Zhang, Shiping Wang, and Carl Yang. 2024. MuseGraph: Graph-oriented instruction tuning of large language models for generic graph mining. arXiv:2403.04780. Retrieved from https://arxiv.org/abs/2403.04780

[175] Yanchao Tan, Zihao Zhou, Hang Lv, Weiming Liu, and Carl Yang. 2024. Walklm: A uniform language model fine-tuning framework for attributed graph embedding. *Advances in Neural Information Processing Systems* 36 (2024).

[176] Jiabin Tang, Yuhao Yang, Wei Wei, Lei Shi, Lixin Su, Suqi Cheng, Dawei Yin, and Chao Huang. 2024. Graphgpt: Graph instruction tuning for large language models. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 491–500.

[177] Yixuan Tang and Yi Yang. 2024. MultiHop-RAG: Benchmarking retrieval-augmented generation for multi-hop queries. arXiv:2401.15391. Retrieved from https://arxiv.org/abs/2401.15391

[178] Dhaval Taunk, Lakshya Khanna, Siri Venkata Pavan Kumar Kandru, Vasudeva Varma, Charu Sharma, and Makarand Tapaswi. 2023. GrapeQA: GRaph augmentation and pruning to enhance question-answering. In *Proceedings of the WWW (Companion Volume)*. ACM, 1138–1144.

[179] S. M. Tonmoy, S. M. Zaman, Vinija Jain, Anku Rani, Vipula Rawte, Aman Chadha, and Amitava Das. 2024. A comprehensive survey of hallucination mitigation techniques in large language models. arXiv:2401.01313. Retrieved from https://arxiv.org/abs/2401.01313

[180] Hugo Touvron, Louis Martin, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. arXiv:2307.09288. Retrieved from https://arxiv.org/abs/2307.09288

[181] P. Veličković, G. Cucurull, A. Casanova, A. Romero,P. Liò, and Y. Bengio. 2018 Graph attention networks. arXiv:1710.10903. Retrieved from https://arxiv.org/abs/1710.10903

[182] Denny Vrandečić and Markus Krötzsch. 2014. Wikidata: A free collaborative knowledgebase. *Communications of the ACM* 57, 10 (2014), 78–85.

[183] Chaojie Wang, Yishi Xu, Zhong Peng, Chenxi Zhang, Bo Chen, Xinrun Wang, Lei Feng, and Bo An. 2023. keqing: knowledge-based question answering is a nature chain-of-thought mentor of LLM. arXiv:2401.00426. Retrieved from https://arxiv.org/abs/2401.00426

[184] Heng Wang, Shangbin Feng, Tianxing He, Zhaoxuan Tan, Xiaochuang Han, and Yulia Tsvetkov. 2024. Can language models solve graph problems in natural language? *Advances in Neural Information Processing Systems* 36 (2024).

[185] Keheng Wang, Feiyu Duan, Sirui Wang, Peiguang Li, Yunsen Xian, Chuantao Yin, Wenge Rong, and Zhang Xiong. 2023. Knowledge-driven CoT: Exploring faithful reasoning in LLMs for knowledge-intensive question answering. arXiv:2308.13259. Retrieved from https://arxiv.org/abs/2308.13259

[186] Xi Wang, Procheta Sen, Ruizhe Li, and Emine Yilmaz. 2024. Adaptive retrieval-augmented generation for conversational systems. *arXiv preprint arXiv:2407.21712* (2024). Retrieved from https://arxiv.org/abs/2407.21712

[187] Xintao Wang, Qianwen Yang, Yongting Qiu, Jiaqing Liang, Qianyu He, Zhouhong Gu, Yanghua Xiao, and Wei Wang. 2023. KnowledGPT: Enhancing large language models with retrieval and storage access on knowledge bases. arXiv:2308.11761. Retrieved from https://arxiv.org/abs/2308.11761

[188] Xiaolong Wang, Yufei Ye, and Abhinav Gupta. 2018. Zero-shot recognition via semantic embeddings and knowledge graphs. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 6857–6866.

[189] Yu Wang, Nedim Lipka, Ryan A. Rossi, Alexa F. Siu, Ruiyi Zhang, and Tyler Derr. 2024. Knowledge graph prompting for multi-document question answering. In *Proceedings of the AAAI*. AAAI Press, 19206–19214.

[190] Yaoke Wang, Yun Zhu, Wenqiao Zhang, Yueting Zhuang, Yunfei Li, and Siliang Tang. 2024. Bridging Local Details and Global Context in Text-Attributed Graphs. arXiv:2406.12608. Retrieved from https://arxiv.org/abs/2406.12608

[191] Yinwei Wei, Xiang Wang, Liqiang Nie, Xiangnan He, Richang Hong, and Tat-Seng Chua. 2019. MMGCN: Multi-modal graph convolution network for personalized recommendation of micro-video. In *Proceedings of the 27th ACM International Conference on Multimedia*. 1437–1445.

[192] Yilin Wen, Zifeng Wang, and Jimeng Sun. 2024. MindMap: Knowledge Graph Prompting Sparks Graph of Thoughts in Large Language Models. arXiv:2308.09729. Retrieved from https://arxiv.org/abs/2308.09729

[193] Junde Wu, Jiayuan Zhu, and Yunli Qi. 2024. Medical Graph RAG: Towards Safe Medical Large Language Model via Graph Retrieval-Augmented Generation. arXiv:2408.04187. Retrieved from https://arxiv.org/abs/2408.04187

[194] Qiming Wu, Zichen Chen, Will Corcoran, Misha Sra, and Ambuj K. Singh. 2024. GraphEval2000: Benchmarking and improving large language models on graph datasets. arXiv:2406.16176. Retrieved from https://arxiv.org/abs/2406.16176

[195] Shangyu Wu, Ying Xiong, Yufei Cui, Haolun Wu, Can Chen, Ye Yuan, Lianming Huang, Xue Liu, Tei-Wei Kuo, Nan Guan, et al. 2024. Retrieval-Augmented Generation for Natural Language Processing: A Survey. arXiv:2407.13193. Retrieved from https://arxiv.org/abs/2407.13193

[196] Shirley Wu, Shiyu Zhao, Michihiro Yasunaga, Kexin Huang, Kaidi Cao, Qian Huang, Vassilis N. Ioannidis, Karthik Subbian, James Zou, and Jure Leskovec. 2024. STaRK: Benchmarking LLM Retrieval on Textual and Relational Knowledge Bases. arXiv:2404.13207. Retrieved from https://arxiv.org/abs/2404.13207

[197] Taiqiang Wu, Xingyu Bai, Weigang Guo, Weijie Liu, Siheng Li, and Yujiu Yang. 2023. Modeling fine-grained information via knowledge-aware hierarchical graph for zero-shot entity retrieval. In *Proceedings of the WSDM*. ACM, 1021–1029.

[198] Yike Wu, Nan Hu, Sheng Bi, Guilin Qi, Jie Ren, Anhuan Xie, and Wei Song. 2023. Retrieve-Rewrite-Answer: A KG-to-Text Enhanced LLMs Framework for Knowledge Graph Question Answering. arXiv:2309.11206. Retrieved from https://arxiv.org/abs/2309.11206

[199] Hao Yan, Chaozhuo Li, Ruosong Long, Chao Yan, Jianan Zhao, Wenwen Zhuang, Jun Yin, Peiyan Zhang, Weihao Han, Hao Sun, et al. 2023. A comprehensive study on text-attributed graphs: Benchmarking and rethinking. *Advances in Neural Information Processing Systems* 36 (2023), 17238–17264.

[200] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. 2024. Corrective retrieval augmented generation. arXiv:2401.15884. Retrieved from https://arxiv.org/abs/2401.15884

[201] Jingfeng Yang, Hongye Jin, Ruixiang Tang, Xiaotian Han, Qizhang Feng, Haoming Jiang, Bing Yin, and Xia Hu. 2023. Harnessing the power of LLMs in practice: A survey on chatgpt and beyond. (2023). arXiv:2304.13712. Retrieved from https://arxiv.org/abs/2304.13712

[202] Rui Yang, Haoran Liu, Edison Marrese-Taylor, Qingcheng Zeng, Yu He Ke, Wanxin Li, Lechao Cheng, Qingyu Chen, James Caverlee, Yutaka Matsuo, et al. 2024. KG-Rank: Enhancing Large Language Models for Medical QA with Knowledge Graphs and Ranking Techniques. arXiv:2403.05881. Retrieved from https://arxiv.org/abs/2403.05881

[203] Rui Yang, Boming Yang, Aosong Feng, Sixun Ouyang, Moritz Blum, Tianwei She, Yuang Jiang, Freddy Lecue, Jinghui Lu, and Irene Li. 2024. Graphusion: A rag framework for knowledge graph construction with a global perspective. arXiv:2410.17600. Retrieved from https://arxiv.org/abs/2410.17600

[204] Xiao Yang, Kai Sun, Hao Xin, Yushi Sun, Nikita Bhalla, Xiangsen Chen, Sajal Choudhary, Rongze Daniel Gui, Ziran Will Jiang, Ziyu Jiang, et al. 2024. CRAG – Comprehensive RAG Benchmark. arXiv:2406.04744. Retrieved from https://arxiv.org/abs/2406.04744

[205] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2369–2380.

[206] Yao Yao, Zuchao Li, and Hai Zhao. 2023. Beyond chain-of-thought, effective graph-of-thought reasoning in language models. arXiv:2305.16582. Retrieved from https://arxiv.org/abs/2305.16582

[207] Michihiro Yasunaga, Hongyu Ren, Antoine Bosselut, Percy Liang, and Jure Leskovec. 2021. QA-GNN: Reasoning with language models and knowledge graphs for question answering. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 535–546.

[208] Xi Ye, Semih Yavuz, Kazuma Hashimoto, Yingbo Zhou, and Caiming Xiong. 2021. Rng-kbqa: Generation augmented iterative ranking for knowledge base question answering. arXiv:2109.08678. Retrieved from https://arxiv.org/abs/2109.08678

[209] Wen-tau Yih, Matthew Richardson, Christopher Meek, Ming-Wei Chang, and Jina Suh. 2016. The value of semantic parse labeling for knowledge base question answering. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. The Association for Computer Linguistics.

[210] Zhitao Ying, Jiaxuan You, Christopher Morris, Xiang Ren, Will Hamilton, and Jure Leskovec. 2018. Hierarchical graph representation learning with differentiable pooling. *Advances in Neural Information Processing Systems* 31 (2018).

[211] Donghan Yu, Sheng Zhang, Patrick Ng, Henghui Zhu, Alexander Hanbo Li, Jun Wang, Yiqun Hu, William Yang Wang, Zhiguo Wang, and Bing Xiang. 2023. DecAF: Joint decoding of answers and logical forms for question answering over knowledge bases. In *ICLR*. OpenReview.net.

[212] Donghan Yu, Chenguang Zhu, Yuwei Fang, Wenhao Yu, Shuohang Wang, Yichong Xu, Xiang Ren, Yiming Yang, and Michael Zeng. 2022. KG-FiD: Infusing knowledge graph in fusion-in-decoder for open-domain question answering. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 4961–4974.

[213] Hao Yu, Aoran Gan, Kai Zhang, Shiwei Tong, Qi Liu, and Zhaofeng Liu. 2024. Evaluation of retrieval-augmented generation: A survey. arXiv:2405.07437. Retrieved from https://arxiv.org/abs/2405.07437

[214] Hanwang Zhang, Zawlin Kyaw, Shih-Fu Chang, and Tat-Seng Chua. 2017. Visual translation embedding network for visual relation detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 5532–5540.

[215] Jiasheng Zhang, Jialin Chen, Ali Maatouk, Ngoc Bui, Qianqian Xie, Leandros Tassiulas, Jie Shao, Hua Xu, and Rex Ying. 2024. LitFM: A retrieval augmented structure-aware foundation model for citation graphs. arXiv:2409.12177. Retrieved from https://arxiv.org/abs/2409.12177

[216] Jing Zhang, Xiaokang Zhang, Jifan Yu, Jian Tang, Jie Tang, Cuiping Li, and Hong Chen. 2022. Subgraph retrieval enhanced model for multi-hop knowledge base question answering. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 5773–5784.

[217] Mengmei Zhang, Mingwei Sun, Peng Wang, Shen Fan, Yanhu Mo, Xiaoxiao Xu, Hong Liu, Cheng Yang, and Chuan Shi. 2024. GraphTranslator: Aligning graph model to large language model for open-ended tasks. In *Proceedings of the ACM on Web Conference 2024, Singapore, May 13-17, 2024*. 1003–1014.

[218] Qinggang Zhang, Shengyuan Chen, Yuanchen Bei, Zheng Yuan, Huachi Zhou, Zijin Hong, Junnan Dong, Hao Chen, Yi Chang, and Xiao Huang. 2025. A survey of graph retrieval-augmented generation for customized large language models. arXiv:2501.13958. Retrieved from https://arxiv.org/abs/2501.13958

[219] Qinggang Zhang, Junnan Dong, Hao Chen, Daochen Zha, Zailiang Yu, and Xiao Huang. 2024. KnowGPT: Knowledge Graph based Prompting for Large Language Models. arXiv:2312.06185. Retrieved from https://arxiv.org/abs/2312.06185

[220] Yuyu Zhang, Hanjun Dai, Zornitsa Kozareva, Alexander Smola, and Le Song. 2018. Variational reasoning for question answering with knowledge graph. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

[221] Yue Zhang, Yafu Li, Leyang Cui, Deng Cai, Lemao Liu, Tingchen Fu, Xinting Huang, Enbo Zhao, Yu Zhang, Yulong Chen, et al. 2023. Siren's song in the AI ocean: A survey on hallucination in large language models. arXiv:2309.01219. Retrieved from https://arxiv.org/abs/2309.01219

[222] Jianan Zhao, Le Zhuo, Yikang Shen, Meng Qu, Kai Liu, Michael Bronstein, Zhaocheng Zhu, and Jian Tang. 2023. GraphText: Graph Reasoning in Text Space. arXiv:2310.01089. Retrieved from https://arxiv.org/abs/2310.01089

[223] Penghao Zhao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, Jie Jiang, and Bin Cui. 2024. Retrieval-Augmented Generation for AI-Generated Content: A Survey. arXiv:2402.19473. Retrieved from https://arxiv.org/abs/2402.19473

[224] Siyun Zhao, Yuqing Yang, Zilong Wang, Zhiyuan He, Luna K. Qiu, and Lili Qiu. 2024. Retrieval Augmented Generation (RAG) and Beyond: A Comprehensive Survey on How to Make your LLMs use External Data More Wisely. arXiv:2409.14924. Retrieved from https://arxiv.org/abs/2409.14924

[225] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. arXiv:2303.18223. Retrieved from https://arxiv.org/abs/2303.18223

[226] Shaowen Zhou, Bowen Yu, Aixin Sun, Cheng Long, Jingyang Li, Haiyang Yu, Jian Sun, and Yongbin Li. 2022. A survey on neural open information extraction: Current status and future directions. arXiv:2205.11725. Retrieved from https://arxiv.org/abs/2205.11725

[227] Xuanhe Zhou, Zhaoyan Sun, and Guoliang Li. 2024. Db-gpt: Large language model meets database. *Data Science and Engineering* 9, 1 (2024), 102–111.

[228] Yujia Zhou, Yan Liu, Xiaoxi Li, Jiajie Jin, Hongjin Qian, Zheng Liu, Chaozhuo Li, Zhicheng Dou, Tsung-Yi Ho, and Philip S. Yu. 2024. Trustworthiness in retrieval-augmented generation systems: A survey. arXiv:2409.10102. Retrieved from https://arxiv.org/abs/2409.10102

[229] Yinghao Zhu, Changyu Ren, Shiyun Xie, Shukai Liu, Hangyuan Ji, Zixiang Wang, Tao Sun, Long He, Zhoujun Li, Xi Zhu, et al. 2024. REALM: RAG-driven enhancement of multimodal electronic health records analysis via large language models. arXiv:2402.07016. Retrieved from https://arxiv.org/abs/2402.07016

[230] Yun Zhu, Yaoke Wang, Haizhou Shi, and Siliang Tang. 2024. Efficient Tuning and Inference for Large Language Models on Textual Graphs. arXiv:2401.15569. Retrieved from https://arxiv.org/abs/2401.15569

[231] Yanxu Zhu, Jinlin Xiao, Yuhang Wang, and Jitao Sang. 2024. KG-FPQ: Evaluating factuality hallucination in llms with knowledge graph-based false premise questions. arXiv:2407.05868. Retrieved from https://arxiv.org/abs/2407.05868

[232] Yutao Zhu, Huaying Yuan, Shuting Wang, Jiongnan Liu, Wenhan Liu, Chenlong Deng, Zhicheng Dou, and Ji-Rong Wen. 2023. Large language models for information retrieval: A survey. arXiv:2308.07107. Retrieved from https://arxiv.org/abs/2308.07107

[233] Zulun Zhu, Haoyu Liu, Mengke He, and Siqiang Luo. 2025. Right answer at the right time-temporal retrieval-augmented generation via graph summarization. arXiv:2510.16715. Retrieved from https://arxiv.org/abs/2510.16715

Received 7 April 2025; revised 11 January 2026; accepted 19 January 2026